```
# 文件 backend/intel_agent/engine/controller.py
"""IntelAgentController: the round loop that ties the agent, memory, and
   rules.
Per round: recall playbook -> render digest -> run an agentic session
   (falling
back to deterministic rules if the SDK is unavailable) -> annotate
   relevance ->
merge -> harvest techniques into the playbook -> recompute coverage -> let
   the
critic decide whether to continue. All heavy collaborators are injectable
   so the
loop is fully testable offline.
"""
from __future__ import annotations
from intel_agent.runtime.client import _setting
import re
import time
from datetime import datetime, timezone
from typing import Any
from ..agent.critic import decide_termination
from ..agent.loop import IntelAgentLoop
from ..agent.system_prompt import render_round_prompt
from ..analysis import ( analyze_coverage_gaps, coverage_quota,
   is_search_stalled, item_is_relevant, merge_batch_into_blackboard,
   min_new_relevant_per_call, new_relevant_per_call, quota_open_gaps,
   stall_patience, )
from ..memory.harvest import harvest_round
from ..memory.playbook import PlaybookStore, default_playbook_path,
   playbook_enabled
from ..memory.retrieval import default_playbook_embedder
from ..observability import TraceSink
from ..persistence import JsonIntelRunStore
from ..relevance import RelevancePipeline, default_pipeline
from ..runtime.structured import StructuredDecisionEngine
from ..store import default_store
from ..schemas import ( ActionDecision, CandidateTopicNamingDecision,
   IntelRunBlackboard, QueryCallOutcome, RawIntelItemBatch, RunGap,
   RunBudget, SearchQueryPlan, SearchReflectionDecision,
   SourceCheckpoint, )
from ..sources import default_registered_api_sources
from ..topics import EXTENDED_SECURITY_TOPICS
from ..tools.context import ToolContext
from .context import render_digest
from .bandit import normalized_reward
from .fallback import run_fallback_round
from .gaps import ( compatibility_coverage_gaps, compute_corpus_gaps,
   detect_candidate_topics, extended_topic_trends, initial_run_gaps,
   total_open_priority, )
from .modes import ( CollectionMode, FullCollectionMode,
   IncrementalCollectionMode, incremental_overlap_hours, )
```

```
DEFAULT_MAX_RESULTS_PER_CALL = 20
def incremental_strategy() -> str:
    value = _setting("INTEL_INCREMENTAL_STRATEGY",
        "adaptive").strip().lower()
    return value if value in {"adaptive", "legacy"} else "adaptive"
class IntelAgentController:
    def __init__( self, store: Any | None = None,
        playbook: PlaybookStore | None = None,
        relevance: RelevancePipeline | None = None,
        loop: IntelAgentLoop | None = None,
        decision_engine: StructuredDecisionEngine | None = None,
        max_results_per_call: int = DEFAULT_MAX_RESULTS_PER_CALL,
        min_rounds: int = 1, ) -> None:
        self.store = store or default_store()
        self.playbook = playbook if playbook is not None else
            _default_playbook()
        self.relevance = relevance if relevance is not None else
            default_pipeline()
        self.loop = loop or IntelAgentLoop()
        self.decision_engine = decision_engine or
            StructuredDecisionEngine()
        self.max_results_per_call = max_results_per_call
        self.min_rounds = min_rounds
    def run( self, mode: CollectionMode | None = None,
        budget: RunBudget | None = None, run_id: str | None = None,
        trace: TraceSink | None = None, ) -> IntelRunBlackboard:
        mode = mode or FullCollectionMode()
        budget = budget or RunBudget(max_rounds=6, max_api_calls=40)
        adaptive = isinstance(mode, IncrementalCollectionMode) and
            incremental_strategy() == "adaptive"
        adaptive_init_error: str | None = None
        self._corpus_items = []
        checkpoints: list[SourceCheckpoint] = []
        self._historical_outcomes: list[QueryCallOutcome] = []
        if adaptive:
            try:
                corpus_loader = getattr(self.store, "load_corpus_items",
                None)
                if callable(corpus_loader):
                    self._corpus_items = list(corpus_loader())
                else:
                    self._corpus_items = [ item
                        for board in
                self.store.load_all_blackboards(limit=200)
                        for item in board.raw_items ]
                checkpoint_loader = getattr(self.store,
                "load_source_checkpoints", None)
                checkpoints = list(checkpoint_loader()) if
                callable(checkpoint_loader) else []
                self._historical_outcomes = [ outcome
```

```
            for board in
        self.store.load_all_blackboards(limit=500)
            for outcome in board.query_outcomes ]
    except Exception as exc:  # noqa: BLE001 - persistence ->
        legacy contract
        adaptive = False
        adaptive_init_error = f"{exc.__class__.__name__}: {exc}"
        self._corpus_items = []
        checkpoints = []
        self._historical_outcomes = []
        if not isinstance(self.store, JsonIntelRunStore):
            self.store = JsonIntelRunStore()
corpus_gaps = compute_corpus_gaps(self._corpus_items) if adaptive
    else []
blackboard = IntelRunBlackboard(
    run_id=run_id or _default_run_id(mode.run_mode),
    run_goal=mode.run_goal,
    run_mode=mode.run_mode,  # type: ignore[arg-type]
    budget=budget,
    approved_sources=default_registered_api_sources(),
    corpus_gaps=corpus_gaps,
    coverage_gaps=compatibility_coverage_gaps(corpus_gaps) if
        adaptive else [],
    run_gaps=initial_run_gaps(corpus_gaps, checkpoints) if
        adaptive else [],
    source_checkpoints=checkpoints,
    extended_trends=extended_topic_trends(self._corpus_items) if
        adaptive else {},
    collection_strategy="adaptive" if adaptive else "legacy", )
if trace is not None:
    trace.bind_run(blackboard.run_id)
if adaptive_init_error:
    blackboard.errors.append(
        _error("adaptive_persistence_failed", adaptive_init_error)
    )
    if trace is not None:
        trace.emit("strategy_fallback",
        reason=adaptive_init_error)
self._batches = []
self._run_started_at = time.monotonic()
self._adaptive_stall_rounds = 0
self._force_rules = False
if self.loop.available():
    self.loop.start_run(blackboard.run_id, continuous=adaptive)
rounds_completed = False
try:
    try:
        self._run_rounds(blackboard, mode, budget, trace)
    except Exception as exc:  # noqa: BLE001 - adaptive run-level
        fallback contract
```

```
            if not adaptive:
                raise
blackboard.errors.append(_error("adaptive_strategy_failed", str(exc)))
            blackboard.collection_strategy = "legacy"
            self._force_rules = True
            if trace is not None:
                trace.emit("strategy_fallback", reason=str(exc)[:300])
            self._run_rounds(blackboard, mode, budget, trace)
         rounds_completed = True
      finally:
         blackboard.sdk_session_id = self.loop.session_id
         self.loop.end_run()
         if not rounds_completed and trace is not None:
            trace.close()
      try:
         blackboard.metrics.round_index = len(blackboard.query_history)
         blackboard.metrics.elapsed_seconds = max(
            0.0, time.monotonic() - self._run_started_at )
         if blackboard.collection_strategy == "adaptive":
            embedder = ( getattr(self.relevance, "embedder", None)
               if self.relevance is not None else None )
            blackboard.candidate_topics = detect_candidate_topics(
               self._corpus_items + blackboard.raw_items, embedder )
            self._credit_candidate_topic_discovery(blackboard)
            self._name_candidate_topics(blackboard)
         status = "succeeded" if blackboard.raw_items else
            "partial_success"
         try:
            self.store.save_run(blackboard, self._batches,
            status=status)
         except Exception as exc:  # noqa: BLE001 - remote store ->
            durable JSON fallback
            if isinstance(self.store, JsonIntelRunStore):
               raise
            blackboard.errors.append(_error("persistence_fallback",
            str(exc)))
            if trace is not None:
               trace.emit("persistence_fallback",
            reason=str(exc)[:300])
            JsonIntelRunStore().save_run(
               blackboard, self._batches, status="partial_success" )
         if self.playbook is not None:
            self.playbook.save()
         return blackboard
      finally:
         if trace is not None:
            trace.close()
   def _name_candidate_topics(self, blackboard: IntelRunBlackboard) ->
      None:
      for candidate in blackboard.candidate_topics:
```

```
            prompt = (
                "Name this deterministically gated emerging LLM-security
                evidence cluster. "
                "Return a short lowercase security topic name and a
                one-sentence rationale. "
                "Do not claim it is part of the formal taxonomy.\n\n"
                f"Current label: {candidate.proposed_name}\n"
                f"Sources: {candidate.source_names}\n"
                f"Supporting item IDs:
                {candidate.supporting_item_ids[:10]}"
            )
            parsed = self.decision_engine.decide( "candidate_topic_name",
                prompt, CandidateTopicNamingDecision,
                system_prompt="You label proposed LLM-security topic
                clusters via submit_decision.",
                fast=True, )
            if parsed is not None:
                candidate.proposed_name =
                parsed.proposed_name.strip().lower()[:120]
                candidate.rationale = parsed.rationale[:500]
    def _credit_candidate_topic_discovery(
        self, blackboard: IntelRunBlackboard ) -> None:
        supporting_ids = { item_id
            for candidate in blackboard.candidate_topics
            for item_id in candidate.supporting_item_ids }
        for outcome in blackboard.query_outcomes:
            if outcome.query_intent != "discovery":
                continue
            new_ids = set(outcome.metadata.get("new_item_ids") or [])
            residual_support = min(1.0, len(new_ids & supporting_ids) /
                3.0)
            outcome.metadata["residual_cluster_support"] =
                round(residual_support, 4)
            outcome.reward = normalized_reward(
                outcome.gap_priority_before, outcome.gap_priority_after,
                outcome, residual_support=residual_support, )
    def _run_rounds( self, blackboard: IntelRunBlackboard,
        mode: CollectionMode, budget: RunBudget, trace: TraceSink | None,
    ) -> None:
        adaptive = blackboard.collection_strategy == "adaptive"
        source_since: dict[str, datetime] = {}
        if adaptive and isinstance(mode, IncrementalCollectionMode):
            source_since, until = mode.resolve_source_scopes( self.store,
                checkpoints=blackboard.source_checkpoints,
                corpus_items=self._corpus_items, )
            since = min(source_since.values()) if source_since else None
        else:
            since, until = mode.resolve_time_scope(self.store)
        quota = coverage_quota()
        max_rounds = budget.max_rounds or 6
```

```
executed_keys: set[str] = set()
if not hasattr(self, "_batches"):
    self._batches = []
engine_available = self.loop.available() and not self._force_rules
if adaptive and not engine_available:
    blackboard.collection_strategy = "legacy"
    blackboard.errors.append(
        _error("adaptive_sdk_unavailable", "SDK runtime
        unavailable; using legacy rules")
    )
    if trace is not None:
        trace.emit("strategy_fallback", reason="SDK runtime
        unavailable")
    self._force_rules = True
    self._run_rounds(blackboard, mode, budget, trace)
    return
for round_index in range(max_rounds):
    ctx = self._build_context(
        blackboard, mode, since, until, executed_keys,
        source_since=source_since
    )
    recall_text = self._recall(mode, blackboard.run_goal)
    open_gaps = (
        list(dict.fromkeys(gap.topic for gap in
        blackboard.corpus_gaps if gap.status == "open"))
        if adaptive
        else quota_open_gaps(blackboard.raw_items,
        mode.target_topics, quota)
    )
    if trace is not None:
        trace.emit(
            "round_start", round=round_index + 1,
        max_rounds=max_rounds,
            mode=blackboard.run_mode, open_gaps=open_gaps, )
    digest = render_digest(
        blackboard, round_index, max_rounds, mode.target_topics,
        quota, recall_text
    )
    prompt = render_round_prompt(
        digest, mode.mode_brief, round_index, max_rounds,
        focus=mode.focus, time_scope_hint=ctx.time_scope_hint(),
        open_gaps=open_gaps,
        max_turns=self.loop.max_turns, )
    used_agent = engine_available
    if engine_available:
        result = self.loop.run_round(ctx, prompt, trace=trace)
        if not result.completed:
            if adaptive:
                blackboard.collection_strategy = "legacy"
                self._force_rules = True
```

```
                    blackboard.errors.append(
                        _error("adaptive_sdk_failed", result.error or
                "no items collected")
                    )
                    if trace is not None:
                        trace.emit( "strategy_fallback",
                            reason=result.error or "adaptive session
                produced no yield",
                        )
                    self._run_rounds(blackboard, mode, budget, trace)
                    return
                used_agent = False
                if trace is not None:
                    trace.emit("session_fallback", reason=result.error
                or "no items collected")
                blackboard.errors.append(
                    _error("agent_round_failed", result.error or "no
                items collected")
                )
            if not used_agent:
                was_adaptive = ctx.adaptive
                ctx.adaptive = False
                run_fallback_round(
                    ctx, blackboard, self._round_plan(mode, round_index),
                mode.target_topics
                )
                ctx.adaptive = was_adaptive
            batch = self._merge_round(blackboard, ctx, mode, round_index)
            self._batches.append(batch)
            self._harvest(ctx, mode, blackboard.run_id)
            if not adaptive:
                blackboard.coverage_gaps = analyze_coverage_gaps(
                    blackboard.raw_items, mode.target_topics, quota )
            if trace is not None:
                last_entry = blackboard.query_history[-1] if
                blackboard.query_history else None
                trace.emit(
                    "round_summary", round=round_index + 1,
                new_items=len(batch.items),
new_relevant=int(last_entry.metadata.get("relevant_new", 0)) if last_entry
                else 0,
                    api_calls_used=blackboard.metrics.api_calls_used,
                    open_gaps=(
                        [gap.topic for gap in blackboard.corpus_gaps if
                gap.status == "open"]
                        if adaptive else
                quota_open_gaps(blackboard.raw_items, mode.target_topics,
                quota)
                    ), )
            should_stop = (
```

```
            self._should_stop_adaptive(blackboard, ctx, round_index,
            max_rounds, digest, trace)
            if adaptive else self._should_stop(
               blackboard, mode, ctx, round_index, max_rounds, quota,
            digest, trace
            ) )
         if should_stop:
            break
   # ------------------------------------------------------------
      per-round
   def _build_context( self, blackboard: IntelRunBlackboard,
      mode: CollectionMode, since: datetime | None,
      until: datetime | None, executed_keys: set[str],
      source_since: dict[str, datetime] | None = None, ) -> ToolContext:
      return ToolContext( run_id=blackboard.run_id,
         target_topics=mode.target_topics,
         topic_bucket=mode.topic_bucket(),
         max_results_per_call=self.max_results_per_call, since=since,
         until=until, source_since=source_since or {},
         existing_item_ids={item.item_id for item in self._corpus_items
            + blackboard.raw_items},
         executed_keys=executed_keys,
         api_calls_used=blackboard.metrics.api_calls_used,
         max_api_calls=blackboard.budget.max_api_calls,
         playbook=self.playbook,
         adaptive=blackboard.collection_strategy == "adaptive",
         corpus_gaps=list(blackboard.corpus_gaps),
         run_gaps=blackboard.run_gaps,
         prior_outcomes=list(self._historical_outcomes) +
            list(blackboard.query_outcomes),
         current_run_outcomes=list(blackboard.query_outcomes),
         source_checkpoints=blackboard.source_checkpoints,
         extended_trends=blackboard.extended_trends,
         extended_focus=bool(mode.focus and mode.focus.strip().lower()
            in EXTENDED_SECURITY_TOPICS),
      )
   def _recall(self, mode: CollectionMode, run_goal: str) -> str | None:
      if self.playbook is None:
         return None
      entries = self.playbook.recall(
         query_text=mode.focus or run_goal,
            topic_bucket=mode.topic_bucket(), top_k=5
      )
      return self.playbook.render_recall(entries) or None
   def _merge_round( self, blackboard: IntelRunBlackboard,
      ctx: ToolContext, mode: CollectionMode, round_index: int,
   ) -> RawIntelItemBatch:
      new_items = list(ctx.collected_items.values())
      prior_run_items = list(blackboard.raw_items)
      if self.relevance is not None and new_items:
```

```
            try:
                self.relevance.annotate(new_items)
            except Exception as exc:  # noqa: BLE001 - relevance is
                best-effort
                blackboard.errors.append(_error("relevance_failed",
                str(exc)))
        batch = RawIntelItemBatch(items=new_items, source_stats=ctx.stats)
        source_names = sorted({call["source_name"] for call in
            ctx.executed_calls})
        query_text = _round_summary_text(ctx) or mode.focus or
            mode.run_goal
        query_plan = SearchQueryPlan( query_text=query_text,
            source_names=source_names, target_topics=mode.target_topics,
            round_index=round_index, )
        entry = merge_batch_into_blackboard(blackboard, query_plan, batch)
        entry.metadata["source_query_plans"] = [ {
                "call_id": call.get("call_id"),
                "candidate_id": call.get("candidate_id"),
                "source_name": call["source_name"],
                "query_text": call.get("query_text", ""),
                "params": call["params"],
                "query_intent": call.get("query_intent", "gap_fill"),
                "target_topics": call.get("target_topics", []),
                "target_gap_ids": call.get("target_gap_ids", []), }
            for call in ctx.executed_calls ]
        if ctx.adaptive:
            self._attribute_adaptive_outcomes(
                blackboard, ctx, prior_run_items, entry )
        blackboard.metrics.api_calls_used = ctx.api_calls_used
        blackboard.metrics.sources_used = len(source_names)
        blackboard.sdk_session_id = self.loop.session_id
        return batch
    def _attribute_adaptive_outcomes( self,
        blackboard: IntelRunBlackboard, ctx: ToolContext,
        prior_run_items: list, entry: Any, ) -> None:
        item_by_id = ctx.collected_items
        running_items = list(self._corpus_items) + list(prior_run_items)
        running_gaps = compute_corpus_gaps(running_items)
        outcomes: list[QueryCallOutcome] = []
        total_raw = total_in_scope = total_duplicates = total_new = 0
        total_relevant = total_noise = 0
        for call in ctx.executed_calls:
            call_items = [item_by_id[item_id] for item_id in
                call.get("new_item_ids", []) if item_id in item_by_id]
            before_gaps = {gap.gap_id: gap for gap in running_gaps}
            before_priority = total_open_priority(running_gaps)
            running_items.extend(call_items)
            running_gaps = compute_corpus_gaps(running_items)
            after_gaps = {gap.gap_id: gap for gap in running_gaps}
            after_priority = total_open_priority(running_gaps)
```

```
relevant_new = sum(1 for item in call_items if
    item_is_relevant(item))
uncertain = sum( 1 for item in call_items
    if isinstance(item.metadata.get("relevance"), dict)
    and item.metadata["relevance"].get("label") == "uncertain"
)
noise = max(0, len(call_items) - relevant_new - uncertain)
extended_new = sum( 1 for item in call_items
    if item_is_relevant(item)
    and set(item.metadata.get("topics") or []) &
    set(EXTENDED_SECURITY_TOPICS)
)
target_gap_ids = list(call.get("target_gap_ids") or [])
filled = [ gap_id for gap_id in target_gap_ids
    if gap_id in before_gaps and gap_id in after_gaps
    and after_gaps[gap_id].priority_score <
    before_gaps[gap_id].priority_score
]
outcome = QueryCallOutcome(
    call_id=str(call.get("call_id") or
    f"{blackboard.run_id}:call:{len(blackboard.query_outcomes)
    + len(outcomes) + 1}"),
    candidate_id=call.get("candidate_id"),
    source_name=call["source_name"],
    query_text=str(call.get("query_text") or ""),
    params=dict(call.get("params") or {}),
    query_intent=call.get("query_intent", "gap_fill"),
    evidence_channel=call.get("evidence_channel",
    "vulnerability_advisory"),
    target_topics=list(call.get("target_topics") or []),
    target_gap_ids=target_gap_ids,
    operator_signature=str(call.get("signature") or ""),
    raw_count=int(call.get("raw_count", len(call_items))),
    in_scope_count=int(call.get("in_scope_count",
    len(call_items))),
    duplicate_count=int(call.get("duplicate_count", 0)),
    new_count=len(call_items), relevant_new=relevant_new,
    noise_count=noise, uncertain_count=uncertain,
    latency_ms=call.get("latency_ms"),
    success=bool(call.get("success", True)),
    error_type=call.get("error_type"),
    has_more=bool(call.get("has_more", False)),
    filled_gap_ids=filled,
    gap_priority_before=round(before_priority, 4),
    gap_priority_after=round(after_priority, 4), metadata={
        "new_item_ids": [item.item_id for item in call_items],
        "extended_new_count": extended_new, }, )
outcome.reward = normalized_reward(before_priority,
    after_priority, outcome)
outcomes.append(outcome)
```

```
            total_raw += outcome.raw_count
            total_in_scope += outcome.in_scope_count
            total_duplicates += outcome.duplicate_count
            total_new += outcome.new_count
            total_relevant += outcome.relevant_new
            total_noise += outcome.noise_count
        blackboard.query_outcomes.extend(outcomes)
        blackboard.corpus_gaps = compute_corpus_gaps(
            list(self._corpus_items) + list(blackboard.raw_items) )
        blackboard.coverage_gaps =
            compatibility_coverage_gaps(blackboard.corpus_gaps)
        blackboard.extended_trends = extended_topic_trends(
            list(self._corpus_items) + list(blackboard.raw_items) )
        entry.result_count = total_raw
        entry.novelty_score = round(total_new / total_raw, 4) if total_raw
            else 0.0
        entry.duplicate_ratio = (
            round(total_duplicates / total_in_scope, 4) if total_in_scope
                else 0.0
        )
        entry.noise_ratio = round(total_noise / total_new, 4) if total_new
            else 0.0
        entry.metadata["relevant_new"] = total_relevant
        entry.metadata["call_outcome_ids"] = [outcome.call_id for outcome
            in outcomes]
        self._update_adaptive_run_state(blackboard, ctx, outcomes)
    def _update_adaptive_run_state( self, blackboard: IntelRunBlackboard,
        ctx: ToolContext, outcomes: list[QueryCallOutcome], ) -> None:
        now = datetime.now(timezone.utc)
        checkpoints = {checkpoint.source_name: checkpoint for checkpoint
            in blackboard.source_checkpoints}
        calls_by_id = {str(call.get("call_id")): call for call in
            ctx.executed_calls}
        for outcome in outcomes:
            checkpoint = checkpoints.get(outcome.source_name) or
                SourceCheckpoint(
                source_name=outcome.source_name )
            checkpoints[outcome.source_name] = checkpoint
            checkpoint.overlap_hours = incremental_overlap_hours()
            checkpoint.last_attempt_at = now
            checkpoint.complete = False
            if outcome.success:
                checkpoint.last_success_at = now
                if outcome.source_name in {"nvd_cve_api", "arxiv_api"} and
                not outcome.has_more:
                    checkpoint.watermark = ctx.until
                    checkpoint.complete = True
                    checkpoint.cursor = None
                elif outcome.source_name == "cisa_kev_json":
                    checkpoint.watermark = ctx.until or now
```

```
            checkpoint.complete = True
    for run_gap in blackboard.run_gaps:
        if run_gap.gap_id in outcome.target_gap_ids and
        outcome.success:
            run_gap.status = "satisfied"
            run_gap.attempts += 1
        if ( run_gap.gap_type == "source_window"
            and run_gap.source_name == outcome.source_name
            and outcome.success and not outcome.has_more ):
            run_gap.status = "satisfied"
            run_gap.attempts += 1
        elif run_gap.gap_type == "discovery" and
        outcome.query_intent == "discovery" and outcome.success:
            run_gap.status = "satisfied"
            run_gap.attempts += 1
    if outcome.has_more:
        offset_key = "nvd_start_index" if outcome.source_name ==
        "nvd_cve_api" else "arxiv_start"
        current_offset = int(outcome.params.get(offset_key) or 0)
        checkpoint.cursor = str(current_offset +
        self.max_results_per_call)
        blackboard.run_gaps.append( RunGap(
               gap_id=f"run:pagination:{outcome.call_id}",
               gap_type="pagination",
               source_name=outcome.source_name,
               evidence_channel=outcome.evidence_channel,
               priority="high", retryable=True,
               rationale="Source page was full; fetch the next
        page.",
               metadata={offset_key: current_offset +
        self.max_results_per_call},
            ) )
    if not outcome.success:
        blackboard.run_gaps.append( RunGap(
               gap_id=f"run:retry:{outcome.call_id}",
               gap_type="retry", source_name=outcome.source_name,
               evidence_channel=outcome.evidence_channel,
               priority="high", retryable=True,
               rationale=outcome.error_type or "source call
        failed",
            ) )
    identifiers: set[str] = set()
    call = calls_by_id.get(outcome.call_id, {})
    call_items = [ ctx.collected_items[item_id]
        for item_id in call.get("new_item_ids", [])
        if item_id in ctx.collected_items ]
    for item in call_items:
        identifiers.update( match.upper() for match in re.findall(
               r"CVE-\d{4}-\d{4,}", f"{item.item_id} {item.title}
        {item.summary}", re.IGNORECASE
```

```
                ) )
        if outcome.query_intent != "confirmation":
            existing_ids = {
                str(gap.metadata.get("identifier") or "") for gap in
            blackboard.run_gaps
            }
            for identifier in sorted(identifiers - existing_ids):
                blackboard.run_gaps.append( RunGap(
                       gap_id=f"run:confirm:{identifier.lower()}",
                       gap_type="entity_confirmation",
                       source_name="cisa_kev_json",
                       evidence_channel="exploitation",
                       priority="medium",
                       rationale=f"Confirm real-world exploitation
            for {identifier}.",
                       metadata={"identifier": identifier}, ) )
        else:
            query_blob = f"{outcome.query_text}
            {outcome.params}".upper()
            for run_gap in blackboard.run_gaps:
                identifier = str(run_gap.metadata.get("identifier") or
            "").upper()
                if run_gap.gap_type == "entity_confirmation" and
            identifier and identifier in query_blob:
                    run_gap.status = "satisfied"
    blackboard.source_checkpoints = list(checkpoints.values())
def _harvest(self, ctx: ToolContext, mode: CollectionMode, run_id:
    str) -> None:
    if self.playbook is None:
        return
    calls = []
    for call in ctx.executed_calls:
        new_relevant = sum( 1 for item_id in call["new_item_ids"]
            if item_id in ctx.collected_items and
            item_is_relevant(ctx.collected_items[item_id])
        )
        calls.append({**call, "new_relevant": new_relevant})
    recorded = {t["source_name"]: t["technique_text"] for t in
        ctx.recorded_techniques if t.get("technique_text")}
    def distiller(call: dict[str, Any]) -> str | None:
        return recorded.get(call["source_name"])
    used_ids = harvest_round(
        self.playbook, run_id, calls, mode.topic_bucket(),
        time_scope_hint=ctx.time_scope_hint(),
        distiller=lambda c: distiller(c) or _fallback_technique(c,
            mode.topic_bucket()),
    )
    self.playbook.decay_unused(set(used_ids))
def _should_stop_adaptive( self, blackboard: IntelRunBlackboard,
    ctx: ToolContext, round_index: int, max_rounds: int, digest: str,
```

```
trace: TraceSink | None = None, ) -> bool:
last_entry = blackboard.query_history[-1] if
   blackboard.query_history else None
yield_per_call = new_relevant_per_call(last_entry) if last_entry
   else 0.0
max_utility = max(ctx.candidate_utilities.values(), default=0.0)
high_retry = any(
   gap.status == "open" and gap.retryable and gap.priority in
      {"high", "critical"}
   for gap in blackboard.run_gaps )
low_yield = yield_per_call < 0.5 and max_utility < 0.05 and not
   high_retry
self._adaptive_stall_rounds = self._adaptive_stall_rounds + 1 if
   low_yield else 0
elapsed = max(0.0, time.monotonic() - self._run_started_at)
stop_reason: str | None = None
if round_index + 1 >= max_rounds:
   stop_reason = "max_rounds"
elif not ctx.budget_remaining():
   stop_reason = "budget_exhausted"
elif blackboard.budget.max_seconds is not None and elapsed >=
   blackboard.budget.max_seconds:
   stop_reason = "timeout"
elif self._adaptive_stall_rounds >= 2:
   stop_reason = "saturated"
features = { "new_relevant_per_call": round(yield_per_call, 4),
   "max_candidate_utility": round(max_utility, 4),
   "stall_rounds": self._adaptive_stall_rounds,
   "open_corpus_gaps": sum(1 for gap in blackboard.corpus_gaps if
      gap.status == "open"),
   "open_run_gaps": sum(1 for gap in blackboard.run_gaps if
      gap.status == "open"),
   "high_priority_retry": high_retry, }
assessment = decide_termination( self.decision_engine, digest,
   features, round_index, max_rounds, self.min_rounds, [],
   stop_reason == "saturated", ctx.budget_remaining(), )
should_continue = stop_reason is None
rationale = ( f"adaptive stop: {stop_reason}" if stop_reason
   else assessment.stop_rationale or "continue while useful
      adaptive queries remain"
)
blackboard.stop_reason = stop_reason
if stop_reason == "saturated":
   for gap in blackboard.corpus_gaps:
      if gap.status == "open":
         gap.status = "exhausted_this_run"
   blackboard.coverage_gaps =
      compatibility_coverage_gaps(blackboard.corpus_gaps)
blackboard.reflection_notes.append( SearchReflectionDecision(
      topics_to_expand=list(
```

```
                dict.fromkeys(gap.topic for gap in
            blackboard.corpus_gaps if gap.status == "open")
            ), rationale=rationale,
            confidence=assessment.completeness_score, ) )
        blackboard.action_history.append( ActionDecision(
            action_type="ASSESS_COLLECTION_YIELD" if should_continue
            else "STOP",
            rationale=rationale, metadata={ "features": features,
                "stop_reason": stop_reason,
                "decision_telemetry":
            self.decision_engine.telemetry.summary(),
            }, ) )
        if trace is not None:
            trace.emit( "critic", should_continue=should_continue,
                completeness_score=assessment.completeness_score,
                rationale=rationale, stop_reason=stop_reason, )
        return not should_continue
    def _should_stop( self, blackboard: IntelRunBlackboard,
        mode: CollectionMode, ctx: ToolContext, round_index: int,
        max_rounds: int, quota: int, digest: str,
        trace: TraceSink | None = None, ) -> bool:
        open_gaps = quota_open_gaps(blackboard.raw_items,
            mode.target_topics, quota)
        stalled = is_search_stalled(blackboard, stall_patience(),
            min_new_relevant_per_call())
        last_entry = blackboard.query_history[-1] if
            blackboard.query_history else None
        features = {
            "new_relevant_per_call":
                round(new_relevant_per_call(last_entry), 3) if last_entry
                else 0.0,
            "duplicate_ratio": round(last_entry.duplicate_ratio, 3) if
                last_entry else 0.0,
            "noise_ratio": round(last_entry.noise_ratio, 3) if last_entry
                else 0.0,
            "open_gaps": open_gaps, }
        assessment = decide_termination(
            self.decision_engine, digest, features, round_index,
                max_rounds, self.min_rounds,
            open_gaps, stalled, ctx.budget_remaining(), )
        if trace is not None:
            trace.emit(
                "critic", should_continue=assessment.should_continue,
                completeness_score=assessment.completeness_score,
                rationale=assessment.stop_rationale or "continue: open
                gaps remain",
            )
        blackboard.reflection_notes.append( SearchReflectionDecision(
            topics_to_expand=open_gaps,
            rationale=assessment.stop_rationale or "continue: open
```

```
                gaps remain",
                confidence=assessment.completeness_score, ) )
        blackboard.action_history.append( ActionDecision(
                action_type="ASSESS_COLLECTION_YIELD" if
                assessment.should_continue else "STOP",
                rationale=assessment.stop_rationale or "continue",
                metadata={ "features": features,
                    "decision_telemetry":
                self.decision_engine.telemetry.summary(),
                }, ) )
        return not assessment.should_continue
    def _round_plan(self, mode: CollectionMode, round_index: int) ->
        SearchQueryPlan:
        return SearchQueryPlan(
            query_text=mode.focus or "", source_names=[s for s in
                _SOURCES],
            target_topics=mode.target_topics, round_index=round_index, )
_SOURCES = ("nvd_cve_api", "arxiv_api", "cisa_kev_json", "osv_dev_api")
def _default_playbook() -> PlaybookStore | None:
    if not playbook_enabled():
        return None
    return PlaybookStore(path=default_playbook_path(),
        embedder=default_playbook_embedder())
def _default_run_id(run_mode: str) -> str:
    stamp = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%SZ")
    return f"real-intel-agent-{run_mode}-{stamp}"
def _round_summary_text(ctx: ToolContext) -> str | None:
    for note in reversed(ctx.notes):
        if isinstance(note, dict) and note.get("summary"):
            return str(note["summary"])
    return None
def _fallback_technique(call: dict[str, Any], topic_bucket: str) -> str:
    from ..memory.harvest import _default_technique_text
    return _default_technique_text(call, topic_bucket)
def _error(error_type: str, message: str):
    from ..schemas import ErrorRecord
    return ErrorRecord(error_type=error_type, message=message[:500])
# 文件 backend/intel_agent/agent/loop.py
"""One agentic collection round = one claude-agent-sdk session.
The agent calls the source/memory/control tools itself inside the SDK's
    native
tool loop; results accumulate in the shared ``ToolContext``. A fresh
    session per
round keeps cost bounded and every decision input (the digest)
    reproducible.
Hard session failures with no collected items degrade to the deterministic
    rules
path; turn-limit / truncated sessions that already gathered items
    soft-complete
so the controller keeps the agent yield instead of re-running rules
```

```
    fallback.
"""
from __future__ import annotations
from intel_agent.runtime.client import _setting
import asyncio
import concurrent.futures
import threading
from dataclasses import dataclass
from typing import Any
from ..observability import TraceSink
from ..runtime import client as runtime_client
from ..runtime.structured import _run_coro_sync
from ..hooks import build_hooks
from ..tools.context import ToolContext
from ..tools.server import build_round_server
from .system_prompt import SYSTEM_PROMPT
DEFAULT_ROUND_TIMEOUT_SECONDS = 300.0
# Search-heavy bootstrap rounds often need many parallel tool batches; 12
    was
# too tight and triggered "Reached maximum number of turns" mid-sweep.
DEFAULT_MAX_TURNS = 24
def resolve_max_turns(explicit: int | None = None) -> int:
    if explicit is not None:
        return max(1, int(explicit))
    raw = (_setting("INTEL_AGENT_MAX_TURNS") or "").strip()
    if raw.isdigit():
        return max(1, int(raw))
    return DEFAULT_MAX_TURNS
@dataclass
class RoundSessionResult:
    completed: bool
    assistant_text: str
    error: str | None = None
class IntelAgentLoop:
    def __init__( self, model: str | None = None,
        max_turns: int | None = None,
        timeout_seconds: float = DEFAULT_ROUND_TIMEOUT_SECONDS,
        runner: Any | None = None, client_factory: Any | None = None,
    ) -> None:
        self.model = model or runtime_client.main_model()
        self.max_turns = resolve_max_turns(max_turns)
        self.timeout_seconds = timeout_seconds
        # Test seam: async (ctx, prompt, model, max_turns) -> str
            (assistant text)
        self._runner = runner
        self._client_factory = client_factory
        self._session: _ContinuousSDKSession | None = None
        self._continuous_enabled = True
    def available(self) -> bool:
        if self._runner is not None or self._client_factory is not None:
```

```
            return True
        ok, _ = runtime_client.sdk_runtime_available()
        return ok
    def start_run(self, run_id: str, continuous: bool = True) -> None:
        if self._runner is not None:
            return
        self.end_run()
        self._continuous_enabled = continuous
        if not continuous:
            return
        self._session = _ContinuousSDKSession( model=self.model,
            max_turns=self.max_turns,
            timeout_seconds=self.timeout_seconds,
            client_factory=self._client_factory, )
    def end_run(self) -> None:
        if self._session is not None:
            self._session.close()
            self._session = None
    @property
    def session_id(self) -> str | None:
        return self._session.session_id if self._session is not None else
            None
    def run_round(
        self, ctx: ToolContext, prompt: str, trace: TraceSink | None =
            None
    ) -> RoundSessionResult:
        if not self.available():
            return RoundSessionResult(False, "", "sdk runtime
                unavailable")
        try:
            if self._runner is None:
                if self._continuous_enabled:
                    if self._session is None:
                        self.start_run(ctx.run_id)
                    assert self._session is not None
                    text = self._session.run_round(ctx, prompt, trace)
                else:
                    text = _run_coro_sync( _one_off_session_runner(
                           ctx, prompt, self.model, self.max_turns, trace
                        ), timeout_seconds=self.timeout_seconds, )
            else:
                text = _run_coro_sync(
                    self._runner(ctx, prompt, self.model, self.max_turns),
                    timeout_seconds=self.timeout_seconds, )
        except Exception as exc:  # noqa: BLE001 - round-level fallback /
            soft-complete
            return _finalize_round(ctx, "", f"{exc.__class__.__name__}:
                {exc}", trace)
        return _finalize_round(ctx, text or "", None, trace)
def _finalize_round( ctx: ToolContext, text: str, error: str | None,
```

```
    trace: TraceSink | None, ) -> RoundSessionResult:
    """Keep agent yield on turn-limit / SDK error if searches already
        wrote items."""
    has_yield = bool(ctx.collected_items) or bool(ctx.notes)
    if error and has_yield:
        _ensure_truncated_note(ctx, error)
        if trace is not None:
            trace.emit( "session_truncated", reason=error,
                collected=len(ctx.collected_items), )
        return RoundSessionResult(True, text, error)
    if error:
        return RoundSessionResult(False, text, error)
    return RoundSessionResult(has_yield, text, None)
def _ensure_truncated_note(ctx: ToolContext, error: str) -> None:
    if ctx.notes:
        return
    ctx.notes.append( { "summary": (
              f"Round truncated ({error[:160]}). Kept
              {len(ctx.collected_items)} "
              "items already collected before the session ended." ),
            "covered_topics": [],
            "remaining_gaps": list(ctx.target_topics), } )
def _short_tool_name(name: str) -> str:
    return name.rsplit("__", 1)[-1] if name.startswith("mcp__") else name
async def _one_off_session_runner( ctx: ToolContext, prompt: str,
    model: str, max_turns: int, trace: TraceSink | None, ) -> str:
    """Legacy/full behavior: one isolated SDK query per controller
        round."""
    from claude_agent_sdk import ( AssistantMessage, ResultMessage,
        TextBlock, ThinkingBlock, ToolResultBlock, ToolUseBlock,
        UserMessage, query, )
    mcp_servers, allowed_tools = build_round_server(ctx)
    options = runtime_client.build_options( model=model,
        max_turns=max_turns, system_prompt=SYSTEM_PROMPT,
        mcp_servers=mcp_servers, allowed_tools=allowed_tools,
        hooks=build_hooks(ctx), )
    texts: list[str] = []
    async for message in query(prompt=prompt, options=options):
        if isinstance(message, AssistantMessage):
            for block in message.content:
                if isinstance(block, TextBlock):
                    texts.append(block.text)
                    if trace and block.text.strip():
                        trace.emit("assistant_text", text=block.text)
                elif isinstance(block, ThinkingBlock) and trace:
                    trace.emit("thinking", text=block.thinking)
                elif isinstance(block, ToolUseBlock) and trace:
                    trace.emit("tool_call",
                tool=_short_tool_name(block.name), input=block.input)
        elif isinstance(message, UserMessage) and trace and
```

```
            isinstance(message.content, list):
            for block in message.content:
                if isinstance(block, ToolResultBlock):
                    trace.emit("tool_result", content=block.content,
                is_error=block.is_error)
        elif isinstance(message, ResultMessage):
            if trace:
                trace.emit( "session_end", turns=message.num_turns,
                    cost_usd=message.total_cost_usd,
                    is_error=message.is_error, )
            if message.is_error and not ctx.collected_items and not
                ctx.notes:
                raise RuntimeError(f"sdk session error:
                {str(message.result)[:200]}")
    return "\n".join(texts)
class _ToolContextProxy:
    """Stable object captured by MCP handlers while the active round
        context changes."""
    def __init__(self) -> None:
        object.__setattr__(self, "_current", None)
    def bind(self, ctx: ToolContext) -> None:
        object.__setattr__(self, "_current", ctx)
    def __getattr__(self, name: str) -> Any:
        current = object.__getattribute__(self, "_current")
        if current is None:
            raise RuntimeError("SDK tool context is not bound to a round")
        return getattr(current, name)
    def __setattr__(self, name: str, value: Any) -> None:
        if name == "_current":
            object.__setattr__(self, name, value)
            return
        current = object.__getattribute__(self, "_current")
        if current is None:
            raise RuntimeError("SDK tool context is not bound to a round")
        setattr(current, name, value)
class _AsyncWorker:
    def __init__(self) -> None:
        self.loop = asyncio.new_event_loop()
        self.thread = threading.Thread(target=self._run,
            name="intel-agent-sdk", daemon=True)
        self.thread.start()
    def _run(self) -> None:
        asyncio.set_event_loop(self.loop)
        self.loop.run_forever()
    def submit(self, coro: Any, timeout: float) -> Any:
        future = asyncio.run_coroutine_threadsafe(coro, self.loop)
        try:
            return future.result(timeout=timeout)
        except concurrent.futures.TimeoutError:
            future.cancel()
```

```
            raise TimeoutError(f"SDK round exceeded {timeout:.0f}s") from
                None
    def close(self) -> None:
        if self.loop.is_running():
            self.loop.call_soon_threadsafe(self.loop.stop)
            self.thread.join(timeout=5)
        if not self.loop.is_closed():
            self.loop.close()
class _ContinuousSDKSession:
    """One ClaudeSDKClient and event loop for the lifetime of an
        incremental run."""
    def __init__( self, model: str, max_turns: int,
        timeout_seconds: float, client_factory: Any | None = None,
    ) -> None:
        self.model = model
        self.max_turns = max_turns
        self.timeout_seconds = timeout_seconds
        self.proxy = _ToolContextProxy()
        self.worker = _AsyncWorker()
        self.client: Any | None = None
        self.session_id: str | None = None
        self.client_factory = client_factory
    def run_round(
        self, ctx: ToolContext, prompt: str, trace: TraceSink | None
    ) -> str:
        self.proxy.bind(ctx)
        return str(self.worker.submit(self._run_with_recovery(prompt,
            trace), self.timeout_seconds))
    async def _connect(self, resume: str | None = None) -> None:
        from claude_agent_sdk import ClaudeSDKClient
        mcp_servers, allowed_tools = build_round_server(self.proxy)  #
            type: ignore[arg-type]
        options = runtime_client.build_options( model=self.model,
            max_turns=self.max_turns, system_prompt=SYSTEM_PROMPT,
            mcp_servers=mcp_servers, allowed_tools=allowed_tools,
            hooks=build_hooks(self.proxy),  # type: ignore[arg-type]
            resume=resume, )
        factory = self.client_factory or ClaudeSDKClient
        self.client = factory(options=options)
        await self.client.connect()
    async def _disconnect(self) -> None:
        if self.client is not None:
            try:
                await self.client.disconnect()
            finally:
                self.client = None
    async def _run_with_recovery(self, prompt: str, trace: TraceSink |
        None) -> str:
        if self.client is None:
            await self._connect()
```

```
        try:
            return await self._query_once(prompt, trace)
        except Exception as original:
            if self.proxy.collected_items or self.proxy.notes:
                raise
            last_error: Exception = original
            recovery_modes = [self.session_id] if self.session_id else []
            recovery_modes.append(None)
            for resume in recovery_modes:
                try:
                    await self._disconnect()
                    await self._connect(resume=resume)
                    if trace is not None:
                        trace.emit("session_recovery", mode="resume" if
            resume else "fresh")
                    return await self._query_once(prompt, trace)
                except Exception as exc:  # noqa: BLE001 - try next
                recovery tier
                    last_error = exc
            raise last_error from original
    async def _query_once(self, prompt: str, trace: TraceSink | None) ->
        str:
        from claude_agent_sdk import ( AssistantMessage, ResultMessage,
            TextBlock, ThinkingBlock, ToolResultBlock, ToolUseBlock,
            UserMessage, )
        assert self.client is not None
        await self.client.query(prompt)
        texts: list[str] = []
        async for message in self.client.receive_response():
            if isinstance(message, AssistantMessage):
                for block in message.content:
                    if isinstance(block, TextBlock):
                        texts.append(block.text)
                        if trace and block.text.strip():
                            trace.emit("assistant_text", text=block.text)
                    elif isinstance(block, ThinkingBlock) and trace:
                        trace.emit("thinking", text=block.thinking)
                    elif isinstance(block, ToolUseBlock) and trace:
                        trace.emit("tool_call",
                tool=_short_tool_name(block.name), input=block.input)
            elif isinstance(message, UserMessage) and trace and
                isinstance(message.content, list):
                for block in message.content:
                    if isinstance(block, ToolResultBlock):
                        trace.emit("tool_result", content=block.content,
                is_error=block.is_error)
            elif isinstance(message, ResultMessage):
                self.session_id = message.session_id or self.session_id
                if trace:
                    trace.emit( "session_end", turns=message.num_turns,
```

```
                    cost_usd=message.total_cost_usd,
                    is_error=message.is_error, )
            # Empty yield + error → hard fail (controller rules fallback).
            # Non-empty yield → return normally; if the SDK also raises
                after this
            # message, run_round soft-completes via _finalize_round.
                if message.is_error and not self.proxy.collected_items and
                not self.proxy.notes:
                    raise RuntimeError(f"sdk session error:
                {str(message.result)[:200]}")
        return "\n".join(texts)
    def close(self) -> None:
        try:
            if self.client is not None:
                self.worker.submit(self._disconnect(), timeout=10)
        finally:
            self.worker.close()
# 文件 backend/intel_agent/runtime/client.py
"""claude-agent-sdk runtime factory for Anthropic-compatible endpoints.
Providers (INTEL_SDK_PROVIDER, auto-detected when unset):
- deepseek: https://api.deepseek.com/anthropic (DeepSeek V4.1 Flash:
    deepseek-flash)
- glm:      https://open.bigmodel.cn/api/anthropic (requires an active
    Coding Plan)
An explicit ANTHROPIC_BASE_URL + ANTHROPIC_AUTH_TOKEN pair overrides both.
All claude_agent_sdk imports are lazy so the package (and the rules-only
fallback) works without the `agentsdk` dependency group installed.
"""
from __future__ import annotations
import os
import re
from dataclasses import dataclass
from functools import lru_cache
from pathlib import Path
from typing import Any
GLM_ANTHROPIC_BASE_URL = "https://open.bigmodel.cn/api/anthropic"
DEEPSEEK_ANTHROPIC_BASE_URL = "https://api.deepseek.com/anthropic"
_ENV_LINE =
re.compile(r"^\s*(?:export\s+)?([A-Za-z_][A-Za-z0-9_]*)\s*=\s*(.*?)\s*$")
@dataclass(frozen=True)
class ProviderSpec:
    name: str
    base_url: str
    key_var: str
    default_main_model: str
    default_fast_model: str
    model_var: str
    fast_model_var: str
PROVIDERS: dict[str, ProviderSpec] = { "deepseek": ProviderSpec(
        name="deepseek", base_url=DEEPSEEK_ANTHROPIC_BASE_URL,
```

```
        key_var="DEEPSEEK_API_KEY", default_main_model="deepseek-flash",
        default_fast_model="deepseek-flash", model_var="DEEPSEEK_MODEL",
        fast_model_var="DEEPSEEK_FAST_MODEL", ), "glm": ProviderSpec(
        name="glm", base_url=GLM_ANTHROPIC_BASE_URL,
        key_var="GLM_API_KEY", default_main_model="glm-4.7",
        default_fast_model="glm-4.5-air", model_var="GLM_MODEL",
        fast_model_var="GLM_FAST_MODEL", ), }
@lru_cache(maxsize=1)
def _dotenv_values() -> dict[str, str]:
    env_path = Path.cwd() / ".env"
    values: dict[str, str] = {}
    if not env_path.is_file():
        return values
    for line in env_path.read_text(encoding="utf-8",
        errors="ignore").splitlines():
        if line.lstrip().startswith("#"):
            continue
        match = _ENV_LINE.match(line)
        if match:
            key, raw = match.groups()
            values[key] = raw.strip().strip('"').strip("'")
    return values
def _setting(name: str, default: str = "") -> str:
    return os.getenv(name) or _dotenv_values().get(name, "") or default
def selected_provider() -> ProviderSpec:
    explicit = _setting("INTEL_SDK_PROVIDER").lower()
    if explicit:
        if explicit not in PROVIDERS:
            raise ValueError(f"INTEL_SDK_PROVIDER={explicit!r} is not one
                of {sorted(PROVIDERS)}")
        return PROVIDERS[explicit]
    for spec in PROVIDERS.values():
        if _setting(spec.key_var):
            return spec
    return PROVIDERS["glm"]
def main_model() -> str:
    spec = selected_provider()
    return _setting("INTEL_SDK_MODEL") or _setting(spec.model_var) or
        spec.default_main_model
def fast_model() -> str:
    spec = selected_provider()
    return _setting("INTEL_SDK_FAST_MODEL") or
        _setting(spec.fast_model_var) or spec.default_fast_model
def api_key() -> str:
    return _setting("ANTHROPIC_AUTH_TOKEN") or
        _setting(selected_provider().key_var)
def base_url() -> str:
    return _setting("ANTHROPIC_BASE_URL") or selected_provider().base_url
def anthropic_env() -> dict[str, str]:
    """Environment injected into the bundled CLI subprocess."""
```

```
    env = { "ANTHROPIC_BASE_URL": base_url(),
        "ANTHROPIC_AUTH_TOKEN": api_key(), "ANTHROPIC_API_KEY": "",
        "ANTHROPIC_DEFAULT_OPUS_MODEL": main_model(),
        "ANTHROPIC_DEFAULT_SONNET_MODEL": main_model(),
        "ANTHROPIC_DEFAULT_HAIKU_MODEL": fast_model(),
        "CLAUDE_CODE_DISABLE_NONESSENTIAL_TRAFFIC": "1",
        "DISABLE_TELEMETRY": "1", }
    # Claude Code refuses --dangerously-skip-permissions
        (bypassPermissions) as
    # root unless it believes it is sandboxed. Docker images often run as
        root.
    sandbox = _setting("IS_SANDBOX")
    if sandbox:
        env["IS_SANDBOX"] = sandbox
    elif hasattr(os, "geteuid") and os.geteuid() == 0:
        env["IS_SANDBOX"] = "1"
    return env
def sdk_runtime_available() -> tuple[bool, str]:
    """Cheap engine-level availability probe (no network call)."""
    if not api_key():
        spec = selected_provider()
        return False, f"missing {spec.key_var} / ANTHROPIC_AUTH_TOKEN
            ({spec.name})"
    try:
        import claude_agent_sdk  # noqa: F401
    except ImportError:
        return False, "claude-agent-sdk not installed (uv sync --group
            agentsdk)"
    return True, "ok"
def build_options( model: str | None = None, max_turns: int = 6,
    system_prompt: str | None = None,
    mcp_servers: dict[str, Any] | None = None,
    allowed_tools: list[str] | None = None,
    hooks: dict[str, Any] | None = None, resume: str | None = None,
) -> Any:
    from claude_agent_sdk import ClaudeAgentOptions
    return ClaudeAgentOptions( tools=[], model=model or main_model(),
        env=anthropic_env(), permission_mode="bypassPermissions",
        max_turns=max_turns, setting_sources=[],
        system_prompt=system_prompt, mcp_servers=mcp_servers or {},
        allowed_tools=allowed_tools or [], hooks=hooks, resume=resume,
        include_hook_events=True, )
# 文件 backend/intel_agent/engine/modes.py
"""Collection modes: full vs incremental, each resolving its own time
    scope.
- FullCollectionMode: every target topic, no time window; maximize
    breadth+depth.
- IncrementalCollectionMode: a focus topic/entity within a time window;
    the lower
  bound defaults to the watermark (max published_at of prior runs) so
```

```
     repeated
  incremental runs only chase genuinely new intel.
"""
from __future__ import annotations
from intel_agent.runtime.client import _setting
from dataclasses import dataclass, field
from datetime import datetime, timedelta, timezone
from ..analysis import topic_bucket
from ..agent.system_prompt import FULL_MODE_BRIEF, INCREMENTAL_MODE_BRIEF
from ..persistence import JsonIntelRunStore
from ..schemas import RawIntelItem, SourceCheckpoint
from ..topics import ALL_SECURITY_TOPICS, CORE_SECURITY_TOPICS
INCREMENTAL_OVERLAP_HOURS = 72
_CHECKPOINTED_SOURCES = ("nvd_cve_api", "arxiv_api")
def incremental_overlap_hours() -> int:
    raw = _setting("INTEL_INCREMENTAL_OVERLAP_HOURS",
        str(INCREMENTAL_OVERLAP_HOURS))
    try:
        return max(0, int(raw))
    except ValueError:
        return INCREMENTAL_OVERLAP_HOURS
@dataclass
class CollectionMode:
    run_mode: str = "bootstrap"
    mode_brief: str = FULL_MODE_BRIEF
    # Coverage / critic targets = Core only. Search may still hit Extended
        via keywords.
    target_topics: list[str] = field(default_factory=lambda:
        list(CORE_SECURITY_TOPICS))
    focus: str | None = None
    run_goal: str = "Collect LLM/AI security intelligence."
    def topic_bucket(self) -> str:
        if self.focus:
            return topic_bucket([self.focus])
        return "general"
    def resolve_time_scope( self, store: JsonIntelRunStore
    ) -> tuple[datetime | None, datetime | None]:
        return None, None
@dataclass
class FullCollectionMode(CollectionMode):
    run_mode: str = "bootstrap"
    mode_brief: str = FULL_MODE_BRIEF
    run_goal: str = (
        "Full collection: maximize relevant LLM/AI security intelligence
            across Core "
        "coverage topics (and related Extended threats) while admitting as
            little "
        "unrelated content as possible." )
@dataclass
class IncrementalCollectionMode(CollectionMode):
```

```
    run_mode: str = "incremental"
    mode_brief: str = INCREMENTAL_MODE_BRIEF
    window_days: int | None = 1
    since: datetime | None = None
    until: datetime | None = None
    use_watermark: bool = True
    def __post_init__(self) -> None:
        if self.focus:
            focus_l = self.focus.strip().lower()
            if focus_l in ALL_SECURITY_TOPICS:
                self.target_topics = [focus_l]
            self.run_goal = (
                f"Incremental collection focused on '{self.focus}' within
                the given time scope."
            )
        else:
            self.run_goal = "Incremental collection within the given time
                scope."
    def resolve_time_scope( self, store: JsonIntelRunStore
    ) -> tuple[datetime | None, datetime | None]:
        until = self.until or datetime.now(timezone.utc)
        if self.since is not None:
            return self.since, until
        window_start = (
            until - timedelta(days=self.window_days) if self.window_days
                else None
        )
        if self.use_watermark:
            watermark = _latest_watermark(store)
            if watermark is not None:
                if window_start is None:
                    return watermark, until
                return max(window_start, watermark), until
        return window_start, until
    def resolve_source_scopes( self, store: JsonIntelRunStore,
        checkpoints: list[SourceCheckpoint] | None = None,
        corpus_items: list[RawIntelItem] | None = None,
    ) -> tuple[dict[str, datetime], datetime]:
        """Resolve independent source cursors with a late-arrival overlap
            window."""
        until = self.until or datetime.now(timezone.utc)
        if self.since is not None:
            return {source: self.since for source in
                _CHECKPOINTED_SOURCES}, until
        fallback = until - timedelta(days=self.window_days or 1)
        checkpoint_list = ( checkpoints if checkpoints is not None
            else getattr(store, "load_source_checkpoints", lambda: [])() )
        checkpoints_by_source = {
            checkpoint.source_name: checkpoint for checkpoint in
                checkpoint_list
```

```
        }
        historical = ( _latest_watermarks_from_items(corpus_items)
            if corpus_items is not None
            else _latest_watermarks_by_source(store) )
        starts: dict[str, datetime] = {}
        for source in _CHECKPOINTED_SOURCES:
            checkpoint = checkpoints_by_source.get(source)
            watermark = checkpoint.watermark if checkpoint and
                checkpoint.complete else historical.get(source)
            starts[source] = (
                watermark - timedelta(hours=incremental_overlap_hours())
                if watermark is not None else fallback )
        return starts, until
def _latest_watermark(store: JsonIntelRunStore) -> datetime | None:
    latest: datetime | None = None
    for blackboard in store.load_all_blackboards(limit=50):
        for item in blackboard.raw_items:
            if item.published_at is None:
                continue
            if latest is None or item.published_at > latest:
                latest = item.published_at
    return latest
def _latest_watermarks_by_source(store: JsonIntelRunStore) -> dict[str,
    datetime]:
    latest: dict[str, datetime] = {}
    for blackboard in store.load_all_blackboards(limit=200):
        for item in blackboard.raw_items:
            if item.published_at is None:
                continue
            current = latest.get(item.source_name)
            if current is None or item.published_at > current:
                latest[item.source_name] = item.published_at
    return latest
def _latest_watermarks_from_items(items: list[RawIntelItem]) -> dict[str,
    datetime]:
    latest: dict[str, datetime] = {}
    for item in items:
        if item.source_name not in _CHECKPOINTED_SOURCES or
            item.published_at is None:
            continue
        prior = latest.get(item.source_name)
        if prior is None or item.published_at > prior:
            latest[item.source_name] = item.published_at
    return latest
# 文件 backend/intel_agent/engine/gaps.py
"""Adaptive incremental gap analysis over the persisted intelligence
    corpus."""
from __future__ import annotations
import hashlib
import re
```

```
from collections import defaultdict
from datetime import datetime, timedelta, timezone
from typing import Callable, Sequence
from ..relevance import cosine_similarity
from ..schemas import ( CandidateTopic, CorpusGap, CoverageGap,
   RawIntelItem, RunGap, SourceCheckpoint, )
from ..topics import ( CORE_SECURITY_TOPICS, EXTENDED_SECURITY_TOPICS,
   TOPIC_ANCHORS, detect_topics, tokenize, )
SOURCE_CHANNEL = { "arxiv_api": "research",
   "nvd_cve_api": "vulnerability_advisory",
   "osv_dev_api": "vulnerability_advisory",
   "cisa_kev_json": "exploitation", }
SOURCE_TRUST = { "arxiv_api": 0.72, "nvd_cve_api": 0.90,
   "osv_dev_api": 0.84, "cisa_kev_json": 0.92, }
CHANNEL_SOURCES = { "research": ["arxiv_api"],
   "vulnerability_advisory": ["nvd_cve_api", "osv_dev_api"],
   "exploitation": ["cisa_kev_json"], }
HALF_LIFE_DAYS = { "research": 365, "vulnerability_advisory": 730,
   "exploitation": 365, }
FRESHNESS_SLA_DAYS = {"research": 180, "vulnerability_advisory": 365}
VULNERABILITY_EXEMPT_TOPICS = {"jailbreak", "model backdoor"}
EmbedFn = Callable[[Sequence[str]], list[list[float]]]
def coverage_targets() -> dict[tuple[str, str], float]:
   targets: dict[tuple[str, str], float] = {}
   for topic in CORE_SECURITY_TOPICS:
      targets[(topic, "research")] = 3.0
      if topic not in VULNERABILITY_EXEMPT_TOPICS:
         targets[(topic, "vulnerability_advisory")] = 3.0
   return targets
def relevance_probability(item: RawIntelItem) -> float:
   relevance = item.metadata.get("relevance")
   if isinstance(relevance, dict):
      if relevance.get("label") == "irrelevant":
         return 0.0
      try:
         return min(1.0, max(0.0, float(relevance.get("score",
            item.relevance_score))))
      except (TypeError, ValueError):
         pass
   return float(item.relevance_score)
def topic_credits(item: RawIntelItem) -> dict[str, float]:
   raw_scores = item.metadata.get("topic_scores")
   scores: dict[str, float] = {}
   if isinstance(raw_scores, dict):
      for topic, value in raw_scores.items():
         if topic not in CORE_SECURITY_TOPICS:
            continue
         try:
            score = min(1.0, max(0.0, float(value)))
         except (TypeError, ValueError):
```

```
            continue
        if score > 0:
            scores[topic] = score
    if not scores:
        topics = set(item.metadata.get("topics") or [])
        relevance = item.metadata.get("relevance")
        if isinstance(relevance, dict) and relevance.get("topic"):
            topics.add(str(relevance["topic"]))
        topics.update(detect_topics(f"{item.title} {item.summary}
            {item.raw_text or ''}"))
        scores = {topic: 1.0 for topic in topics if topic in
            CORE_SECURITY_TOPICS}
    total = sum(scores.values())
    if total > 1.0:
        return {topic: score / total for topic, score in scores.items()}
    return scores
def canonical_event_key(item: RawIntelItem) -> str:
    text = f"{item.item_id} {item.title} {item.summary}"
    match = re.search(
r"(CVE-\d{4}-\d{4,}|GHSA-[a-z0-9-]{14}|PYSEC-\d{4}-\d+|RUSTSEC-\d{4}-\d+)"
,
        text, re.IGNORECASE, )
    if match:
        return match.group(1).upper()
    content = " ".join(f"{item.title} {item.summary}".lower().split())
    return hashlib.sha256(content.encode("utf-8")).hexdigest()[:24]
def _uniqueness_weights(items: list[RawIntelItem]) -> dict[str, float]:
    weights: dict[str, float] = {}
    seen_ids: set[str] = set()
    event_sources: dict[str, set[str]] = defaultdict(set)
    ordered = sorted(items, key=lambda item: item.published_at or
        item.fetched_at)
    for item in ordered:
        if item.item_id in seen_ids:
            weights[item.item_id] = 0.0
            continue
        seen_ids.add(item.item_id)
        key = canonical_event_key(item)
        sources = event_sources[key]
        if not sources:
            weight = 1.0
        elif item.source_name not in sources:
            weight = 0.5
        else:
            weight = 0.0
        sources.add(item.source_name)
        weights[item.item_id] = weight
    return weights
def freshness_weight(item: RawIntelItem, channel: str, now: datetime) ->
    float:
```

```
            if max(cosine_similarity(vector, item_vectors[other]) for
                other in cluster) >= 0.82:
                cluster.append(index)
                break
        else:
            clusters.append([index])
    candidates: list[CandidateTopic] = []
    for number, cluster in enumerate(clusters, start=1):
        cluster_items = [residual[index] for index in cluster]
        independent_items: dict[str, RawIntelItem] = {}
        for item in cluster_items:
            independent_items.setdefault(canonical_event_key(item), item)
        sources = sorted({item.source_name for item in cluster_items})
        mean_relevance = sum(relevance_probability(item) for item in
            cluster_items) / len(cluster_items)
        max_anchor = max(
            (cosine_similarity(item_vectors[index], anchor) for index in
                cluster for anchor in anchor_vectors),
            default=1.0, )
        recent_count = 0
        baseline_count = 0
        for item in independent_items.values():
            observed = item.published_at or item.fetched_at
            if observed.tzinfo is None:
                observed = observed.replace(tzinfo=timezone.utc)
            age = now - observed
            if age <= timedelta(days=30):
                recent_count += 1
            elif age <= timedelta(days=120):
                baseline_count += 1
        monthly_baseline = baseline_count / 3.0
        burst_ratio = recent_count / max(0.5, monthly_baseline)
        if ( len(independent_items) < 3 or len(sources) < 2
            or mean_relevance < 0.7 or max_anchor >= 0.7
            or recent_count < 2 or burst_ratio < 1.5 ):
            continue
        tokens: dict[str, int] = defaultdict(int)
        for item in cluster_items:
            for token in tokenize(f"{item.title} {item.summary}"):
                tokens[token] += 1
        label = " ".join(token for token, _ in sorted(tokens.items(),
            key=lambda row: (-row[1], row[0]))[:4])
        support = min(1.0, len(independent_items) / 5)
        diversity = min(1.0, len(sources) / 3)
        separation = min(1.0, max(0.0, (1.0 - max_anchor) / 0.3))
        burst = min(1.0, burst_ratio / 4.0)
        score = min(1.0, support * diversity * separation * burst *
            mean_relevance)
        item_ids = sorted(item.item_id for item in cluster_items)
        digest =
```

```
hashlib.sha256("|".join(item_ids).encode("utf-8")).hexdigest()[:12]
       candidates.append( CandidateTopic(
              candidate_id=f"candidate:{digest}",
              proposed_name=label or f"emerging cluster {number}",
              supporting_item_ids=item_ids, source_names=sources,
              mean_relevance=round(mean_relevance, 4),
              max_anchor_similarity=round(max_anchor, 4),
              emergence_score=round(score, 4), rationale=(
                  "Residual evidence passed independent-support,
              source-diversity, "
                  "relevance, separation, and temporal-burst gates." ),
              metadata={
                  "independent_evidence_count": len(independent_items),
                  "recent_30d": recent_count,
                  "baseline_monthly": round(monthly_baseline, 3),
                  "burst_ratio": round(burst_ratio, 3), }, ) )
    return candidates
# 文件 backend/intel_agent/engine/bandit.py
"""Deterministic UCB ranking and reward attribution for adaptive
    queries."""
from __future__ import annotations
import math
from collections import defaultdict
from ..analysis import operator_signature
from ..schemas import CorpusGap, QueryCallOutcome, QueryCandidate
def arm_key(candidate: QueryCandidate) -> str:
    topic = candidate.target_topics[0] if candidate.target_topics else
        "general"
    return "|".join( [ topic, candidate.evidence_channel,
           candidate.source_name, candidate.query_intent,
           operator_signature(candidate.source_name, candidate.params), ]
    )
def _outcome_arm(outcome: QueryCallOutcome) -> str:
    topic = outcome.target_topics[0] if outcome.target_topics else
        "general"
    return "|".join(
        [topic, outcome.evidence_channel, outcome.source_name,
           outcome.query_intent, outcome.operator_signature]
    )
def score_candidates( candidates: list[QueryCandidate],
    outcomes: list[QueryCallOutcome], corpus_gaps: list[CorpusGap],
) -> list[tuple[QueryCandidate, float]]:
    stats: dict[str, list[float]] = defaultdict(list)
    for outcome in outcomes:
        stats[_outcome_arm(outcome)].append(float(outcome.reward))
    total_calls = len(outcomes)
    gaps = {gap.gap_id: gap for gap in corpus_gaps}
    ranked: list[tuple[QueryCandidate, float]] = []
    for candidate in candidates:
        rewards = stats.get(arm_key(candidate), [])
```

```
        history_mean = sum(rewards) / len(rewards) if rewards else 0.0
        target_priority = sum( gaps[gap_id].priority_score
            for gap_id in candidate.target_gap_ids
            if gap_id in gaps and gaps[gap_id].status == "open" )
        if not candidate.target_gap_ids and candidate.query_intent ==
            "discovery":
            target_priority = 0.2
        expected_reduction = min(1.0, 0.6 * target_priority + 0.4 *
            max(0.0, history_mean))
        exploration = math.sqrt(2.0 * math.log(total_calls + 2) /
            (len(rewards) + 1))
        historical = [outcome for outcome in outcomes if
            _outcome_arm(outcome) == arm_key(candidate)]
        noise = (
            sum(outcome.noise_count / max(1, outcome.new_count) for
                outcome in historical) / len(historical)
            if historical else 0.0 )
        duplicate = (
            sum(outcome.duplicate_count / max(1, outcome.in_scope_count)
                for outcome in historical)
            / len(historical) if historical else 0.0 )
        utility = expected_reduction - 0.2 * noise - 0.2 * duplicate +
            exploration
        ranked.append((candidate, round(utility, 4)))
    return sorted(ranked, key=lambda row: (-row[1], row[0].candidate_id))
def select_portfolio( candidates: list[QueryCandidate],
    outcomes: list[QueryCallOutcome], corpus_gaps: list[CorpusGap],
    max_calls: int = 4, max_total_calls: int | None = None,
    discovery_unbounded: bool = False,
    budget_outcomes: list[QueryCallOutcome] | None = None,
) -> list[tuple[QueryCandidate, float]]:
    ranked = score_candidates(candidates, outcomes, corpus_gaps)
    selected: list[tuple[QueryCandidate, float]] = []
    total_budget = max_total_calls or max(1, len(outcomes) + max_calls)
    prior_counts: dict[str, int] = defaultdict(int)
    for outcome in budget_outcomes if budget_outcomes is not None else
        outcomes:
        prior_counts[outcome.query_intent] += 1
    discovery_remaining = max(0, math.ceil(total_budget * 0.20) -
        prior_counts["discovery"])
    confirmation_remaining = max(0, math.ceil(total_budget * 0.10) -
        prior_counts["confirmation"])
    intent_caps = {
        "discovery": max_calls if discovery_unbounded else min(1,
            discovery_remaining),
        "confirmation": min(1, confirmation_remaining), }
    intent_counts: dict[str, int] = defaultdict(int)
    for row in ranked:
        candidate = row[0]
        cap = intent_caps.get(candidate.query_intent)
```

```
        if cap is not None and intent_counts[candidate.query_intent] >=
            cap:
            continue
        selected.append(row)
        intent_counts[candidate.query_intent] += 1
        if len(selected) >= max_calls:
            break
    return selected
def normalized_reward( before_priority: float, after_priority: float,
    outcome: QueryCallOutcome, residual_support: float = 0.0, ) -> float:
    denominator = max(1.0, before_priority)
    gap_reduction = max(0.0, before_priority - after_priority) /
        denominator
    noise_ratio = outcome.noise_count / max(1, outcome.new_count)
    duplicate_ratio = outcome.duplicate_count / max(1,
        outcome.in_scope_count)
    discovery_bonus = 0.0
    if outcome.query_intent == "discovery":
        extended_new = int(outcome.metadata.get("extended_new_count") or
            0)
        discovery_bonus = min(0.25, 0.05 * extended_new + 0.1 *
            residual_support)
    confirmation_bonus = ( min(0.25, 0.1 * outcome.relevant_new)
        if outcome.query_intent == "confirmation" and outcome.success
        else 0.0 )
    reward = ( gap_reduction + discovery_bonus + confirmation_bonus
        - 0.2 * noise_ratio - 0.2 * duplicate_ratio - 0.05 )
    return round(max(-1.0, min(1.0, reward)), 4)
# 文件 backend/intel_agent/tools/adaptive_tools.py
"""Planning tools that keep LLM proposals behind deterministic scoring."""
from __future__ import annotations
import json
from typing import Any
from pydantic import ValidationError
from ..engine.bandit import select_portfolio
from ..engine.gaps import SOURCE_CHANNEL
from ..schemas import QueryCandidate
from ..topics import ALL_SECURITY_TOPICS
from .context import ToolContext
_STATE_SCHEMA = {"type": "object", "properties": {}}
_CANDIDATE_SCHEMA = { "type": "object", "properties": { "candidates": {
          "type": "array", "maxItems": 8, "items": { "type": "object",
              "properties": { "candidate_id": {"type": "string"},
                  "source_name": { "type": "string",
                      "enum": ["nvd_cve_api", "arxiv_api",
              "cisa_kev_json", "osv_dev_api"],
                  }, "query_text": {"type": "string"}, "params": {
                      "type": "object", "description": (
                          "All source-specific operators must be nested
              here, e.g. "
```

```
                    "arxiv_search_query, nvd_cwe_id,
            osv_ecosystem, or cisa_keyword."
                   ), }, "query_intent": { "type": "string",
                   "enum": ["gap_fill", "discovery", "entity_expand",
            "confirmation"],
                }, "evidence_channel": { "type": "string",
                   "enum": ["research", "vulnerability_advisory",
            "exploitation"],
                },
                "target_topics": {"type": "array", "items": {"type":
            "string"}},
                "target_gap_ids": {"type": "array", "items": {"type":
            "string"}},
                "rationale": {"type": "string"}, }, "required": [
                "candidate_id", "source_name", "query_text",
            "query_intent",
                "evidence_channel", "target_topics", "target_gap_ids",
            ], }, } }, "required": ["candidates"], }
def build_adaptive_tools(ctx: ToolContext) -> list[Any]:
    from claude_agent_sdk import tool
    @tool( "get_collection_state",
        "Return the authoritative Core CorpusGaps, current RunGaps, source
            checkpoints, and "
        "recent per-call rewards. Extended topics never appear as coverage
            gaps.",
        _STATE_SCHEMA, )
    async def get_collection_state(args: dict[str, Any]) -> dict[str,
        Any]:
        state = { "corpus_gaps": [ gap.model_dump(mode="json")
                for gap in ctx.corpus_gaps if gap.status == "open" ][:24],
            "run_gaps": [gap.model_dump(mode="json") for gap in
                ctx.run_gaps if gap.status == "open"],
            "source_checkpoints": [checkpoint.model_dump(mode="json") for
                checkpoint in ctx.source_checkpoints],
            "extended_trends": ctx.extended_trends,
            "historical_reward_summary": _historical_reward_summary(ctx),
            "api_calls_remaining": (
                None if ctx.max_api_calls is None else max(0,
                ctx.max_api_calls - ctx.api_calls_used)
            ), }
        return _text(json.dumps(state, ensure_ascii=False))
    @tool( "score_query_candidates",
        "Submit 1-8 source-specific query candidates. Deterministic UCB
            scoring approves at "
        "most four. Only approved candidate IDs may be passed to source
            tools.",
        _CANDIDATE_SCHEMA, )
    async def score_query_candidates(args: dict[str, Any]) -> dict[str,
        Any]:
        raw = list(args.get("candidates") or [])[:8]
```

```
        candidates: list[QueryCandidate] = []
        errors: list[str] = []
        seen: set[str] = set()
        for entry in raw:
            try:
                candidate = QueryCandidate.model_validate(entry)
            except ValidationError as exc:
                errors.append(str(exc.errors(include_url=False))[:300])
                continue
            if candidate.candidate_id in seen:
                errors.append(f"duplicate candidate_id:
                {candidate.candidate_id}")
                continue
            validation_error = candidate_validation_error(candidate, ctx)
            if validation_error:
                errors.append(f"{candidate.candidate_id}:
                {validation_error}")
                continue
            seen.add(candidate.candidate_id)
            candidates.append(candidate)
        remaining = ( ctx.max_calls_this_round
            if ctx.max_api_calls is None
            else min(ctx.max_calls_this_round, max(0, ctx.max_api_calls -
                ctx.api_calls_used))
        )
        selected = select_portfolio( candidates, ctx.prior_outcomes,
            ctx.corpus_gaps, max_calls=remaining,
            max_total_calls=ctx.max_api_calls,
            discovery_unbounded=ctx.extended_focus,
            budget_outcomes=ctx.current_run_outcomes, )
        ctx.approved_candidates = {candidate.candidate_id: candidate for
            candidate, _ in selected}
        ctx.candidate_utilities = {candidate.candidate_id: utility for
            candidate, utility in selected}
        payload = { "approved": [
                {**candidate.model_dump(mode="json"), "utility": utility}
                for candidate, utility in selected ],
            "rejected_count": len(candidates) - len(selected),
            "validation_errors": errors, }
        return _text(json.dumps(payload, ensure_ascii=False))
    return [get_collection_state, score_query_candidates]
def candidate_validation_error(
    candidate: QueryCandidate, ctx: ToolContext ) -> str | None:
    from .source_tools import candidate_exceeds_time_scope
    if candidate.evidence_channel !=
        SOURCE_CHANNEL[candidate.source_name]:
        return "source and evidence_channel do not match"
    unknown_topics = sorted(set(candidate.target_topics) -
        set(ALL_SECURITY_TOPICS))
    if unknown_topics:
```

```
        return f"unknown taxonomy topics: {unknown_topics}"
    open_gap_ids = {
        gap.gap_id for gap in ctx.corpus_gaps if gap.status == "open"
    } | {gap.gap_id for gap in ctx.run_gaps if gap.status == "open"}
    invalid_gap_ids = sorted(set(candidate.target_gap_ids) - open_gap_ids)
    if invalid_gap_ids:
        return f"target gaps are not open: {invalid_gap_ids}"
    if candidate_exceeds_time_scope(
        ctx, candidate.source_name, candidate.params ):
        return "candidate time operators exceed the authoritative
            incremental window"
    if candidate.query_intent == "gap_fill" and not
        candidate.target_gap_ids:
        return "gap_fill candidates require an open target gap"
    return None
def _historical_reward_summary(ctx: ToolContext) -> list[dict[str, Any]]:
    arms: dict[tuple[str, str, str, str, str], dict[str, float]] = {}
    for outcome in ctx.prior_outcomes:
        topic = outcome.target_topics[0] if outcome.target_topics else
            "general"
        key = ( topic, outcome.evidence_channel, outcome.source_name,
            outcome.query_intent, outcome.operator_signature, )
        row = arms.setdefault( key,
            {"calls": 0.0, "reward": 0.0, "noise": 0.0, "duplicate": 0.0},
        )
        row["calls"] += 1
        row["reward"] += outcome.reward
        row["noise"] += outcome.noise_count / max(1, outcome.new_count)
        row["duplicate"] += outcome.duplicate_count / max(1,
            outcome.in_scope_count)
    summary = []
    for key, totals in arms.items():
        calls = int(totals["calls"])
        summary.append( { "arm": key, "calls": calls,
               "mean_reward": round(totals["reward"] / calls, 4),
               "mean_noise_ratio": round(totals["noise"] / calls, 4),
               "mean_duplicate_ratio": round(totals["duplicate"] / calls,
               4),
            } )
    return sorted( summary,
        key=lambda row: (-row["mean_reward"], -row["calls"], row["arm"]),
    )[:12]
def _text(message: str) -> dict[str, Any]:
    return {"content": [{"type": "text", "text": message}]}
# 文件 backend/intel_agent/tools/source_tools.py
"""Source-search tools the agent calls directly inside a round.
Each registered source is a typed ``@tool`` exposing its advanced
    operators, so
the agent learns to combine them (this is the roadmap's "high-level search
strategy" capability). Handlers are thin wrappers over
```

```
    ``sources.search_source``;
they enforce the API-call budget and cross-round query dedup, apply the
    active
time scope (incremental mode), and record every call for playbook
    harvesting.
"""
from __future__ import annotations
import json
import re
from datetime import datetime, timezone
from typing import Any
from ..analysis import operator_signature, source_query_key,
    SourceQuerySpec
from ..engine.gaps import SOURCE_CHANNEL
from ..sources import build_arxiv_search_query, search_source
from .context import ToolContext
_NVD_SCHEMA = { "type": "object", "properties": {
       "query_text": {"type": "string", "description": "Free-text mission
           query / keywordSearch."},
       "nvd_cwe_id": {"type": "string", "description": "CWE filter, e.g.
           CWE-94, CWE-502."},
       "nvd_cvss_v3_severity": {"type": "string", "description": "HIGH or
           CRITICAL."},
       "nvd_pub_start_date": {"type": "string", "description": "ISO-8601
           ext, e.g. 2026-07-01T00:00:00.000"},
       "nvd_pub_end_date": {"type": "string", "description": "ISO-8601
           ext; pair with start."},
       "nvd_has_kev": {"type": "boolean", "description": "Only CVEs in
           the CISA KEV catalog."},
       "nvd_keyword_exact_match": {"type": "boolean", "description":
           "Phrase match for multi-word keywords."},
       "nvd_cve_id": {"type": "string", "description": "Direct CVE
           lookup."},
       "nvd_start_index": {"type": "integer", "minimum": 0,
           "description": "NVD page offset."},
       "candidate_id": {"type": "string", "description": "Approved
           adaptive candidate ID."},
   }, "required": ["query_text"], }
_ARXIV_SCHEMA = { "type": "object", "properties": {
       "query_text": {"type": "string", "description": "Mission query
           (used if no arxiv_search_query)."},
       "arxiv_search_query": { "type": "string",
           "description": 'arXiv query language: all:/ti:/abs:, AND/OR,
              quoted phrases, cat:cs.CR. '
           'E.g. (all:"prompt injection") AND (cat:cs.CR)', },
       "arxiv_start": {"type": "integer", "minimum": 0, "description":
           "arXiv page offset."},
       "candidate_id": {"type": "string", "description": "Approved
           adaptive candidate ID."},
   }, "required": ["query_text"], }
```

```
_CISA_SCHEMA = { "type": "object", "properties": {
       "query_text": {"type": "string", "description": "Keyword to match
          against KEV entries."},
       "cisa_keyword": {"type": "string", "description": "Explicit
          keyword override."},
       "cisa_cve_ids": {"type": "array", "items": {"type": "string"},
          "description": "Specific CVE IDs."},
       "candidate_id": {"type": "string", "description": "Approved
          adaptive candidate ID."},
   }, "required": ["query_text"], }
_OSV_SCHEMA = { "type": "object", "properties": {
       "query_text": {"type": "string", "description": "Mission query /
          identifier."},
       "osv_ecosystem": {"type": "string", "description": "e.g. PyPI,
          npm, Go."},
       "osv_package_name": {"type": "string", "description": "Package to
          query (pair with ecosystem)."},
       "osv_purl": {"type": "string", "description": "Package URL."},
       "osv_vuln_id": {"type": "string", "description":
          "OSV/GHSA/PYSEC/CVE id for direct lookup."},
       "candidate_id": {"type": "string", "description": "Approved
          adaptive candidate ID."},
   }, "required": ["query_text"], }
_PARAM_KEYS: dict[str, tuple[str, ...]] = { "nvd_cve_api": (
       "nvd_cwe_id", "nvd_cvss_v3_severity", "nvd_pub_start_date",
          "nvd_pub_end_date",
       "nvd_has_kev", "nvd_keyword_exact_match", "nvd_cve_id",
       "nvd_start_index", ),
   "arxiv_api": ("arxiv_search_query", "arxiv_start"),
   "cisa_kev_json": ("cisa_keyword", "cisa_cve_ids"),
   "osv_dev_api": ("osv_ecosystem", "osv_package_name", "osv_purl",
       "osv_vuln_id"),
}
def validate_adaptive_candidate_call(
   ctx: ToolContext, source_name: str, candidate_id: str ) -> str | None:
   """Apply the hook/handler shared adaptive-query safety checks."""
   if not ctx.adaptive:
       return None
   candidate = ctx.approved_candidates.get(candidate_id)
   if candidate is None:
       return "Adaptive source calls require an approved candidate_id."
   if candidate.source_name != source_name:
       return f"Candidate {candidate.candidate_id} is for
          {candidate.source_name}."
   if candidate.evidence_channel != SOURCE_CHANNEL[source_name]:
       return "Candidate evidence_channel does not match the selected
          source."
   if candidate_exceeds_time_scope(ctx, source_name, candidate.params):
       return "Candidate time operators exceed the authoritative
          incremental window."
```

```
    params = { key: value for key, value in candidate.params.items()
        if key in _PARAM_KEYS[source_name] and value not in (None, "", [],
            {})
    }
    _apply_time_scope(source_name, params, ctx)
    spec = SourceQuerySpec( source_name=source_name,
        query_text=candidate.query_text,
        target_topics=list(candidate.target_topics),
        max_results=ctx.max_results_per_call, strategy_name="agent",
        params=params, )
    if source_query_key(spec) in ctx.executed_keys:
        return "This exact approved query already ran."
    if len(ctx.executed_calls) >= ctx.max_calls_this_round:
        return "Per-round source-call budget exhausted."
    if not ctx.budget_remaining():
        return "API-call budget exhausted for this run."
    return None
def candidate_exceeds_time_scope(
    ctx: ToolContext, source_name: str, params: dict[str, Any] ) -> bool:
    since, until = ctx.source_time_scope(source_name)
    if since is None:
        return False
    if source_name == "nvd_cve_api":
        start = _parse_time_operator(params.get("nvd_pub_start_date"))
        end = _parse_time_operator(params.get("nvd_pub_end_date"))
        return bool((start and start < since) or (until and end and end >
            until))
    if source_name == "arxiv_api":
        query = str(params.get("arxiv_search_query") or "")
        match = re.search(
            r"submittedDate:\[(\d{12})\s+TO\s+(\d{12})\]", query,
                re.IGNORECASE
        )
        if not match:
            return False
        start = datetime.strptime(match.group(1),
            "%Y%m%d%H%M").replace(tzinfo=timezone.utc)
        end = datetime.strptime(match.group(2),
            "%Y%m%d%H%M").replace(tzinfo=timezone.utc)
        return start < since or bool(until and end > until)
    return False
def _parse_time_operator(value: Any) -> datetime | None:
    if not value:
        return None
    try:
        parsed = datetime.fromisoformat(str(value).replace("Z", "+00:00"))
    except ValueError:
        return None
    return parsed.replace(tzinfo=parsed.tzinfo or
        timezone.utc).astimezone(timezone.utc)
```

```
def _apply_time_scope(source_name: str, params: dict[str, Any], ctx:
    ToolContext) -> None:
    """Inject native date operators for sources that support them
        (incremental mode)."""
    since, until = ctx.source_time_scope(source_name)
    if since is None:
        return
    if source_name == "nvd_cve_api" and (ctx.adaptive or
        "nvd_pub_start_date" not in params):
        params["nvd_pub_start_date"] =
            since.strftime("%Y-%m-%dT%H:%M:%S.000")
        end = until if until is not None else datetime.now(timezone.utc)
        params["nvd_pub_end_date"] = end.strftime("%Y-%m-%dT%H:%M:%S.000")
    elif source_name == "arxiv_api":
        existing = params.get("arxiv_search_query") or ""
        if ctx.adaptive and "submittedDate" in existing:
            existing = re.sub( r"\s+AND\s+submittedDate:\[[^\]]+\]", "",
                str(existing), flags=re.IGNORECASE, ).strip()
        if "submittedDate" not in existing:
            frm = since.strftime("%Y%m%d%H%M")
            to = until.strftime("%Y%m%d%H%M") if until is not None else
                since.strftime("%Y%m%d") + "2359"
            window = f"submittedDate:[{frm} TO {to}]"
            params["arxiv_search_query"] = f"({existing}) AND {window}" if
                existing else window
def _run_call(ctx: ToolContext, source_name: str, args: dict[str, Any]) ->
    dict[str, Any]:
    query_text = str(args.get("query_text") or "").strip()
    candidate_id = str(args.get("candidate_id") or "").strip() or None
    params = { key: args[key] for key in _PARAM_KEYS[source_name]
        if key in args and args[key] not in (None, "", [], {}) }
    query_intent = "gap_fill"
    target_topics = list(ctx.target_topics)
    target_gap_ids: list[str] = []
    evidence_channel = SOURCE_CHANNEL[source_name]
    if ctx.adaptive:
        denial = validate_adaptive_candidate_call(ctx, source_name,
            candidate_id or "")
        if denial:
            return _text(f"Denied: {denial}")
        candidate = ctx.approved_candidates[candidate_id or ""]
        query_text = candidate.query_text
        params = { key: value for key, value in candidate.params.items()
            if key in _PARAM_KEYS[source_name] and value not in (None, "",
                [], {})
        }
        query_intent = candidate.query_intent
        target_topics = list(candidate.target_topics)
        target_gap_ids = list(candidate.target_gap_ids)
    if source_name == "arxiv_api" and not
```

```
        params.get("arxiv_search_query"):
        params["arxiv_search_query"] = build_arxiv_search_query(
            query_text, target_topics or ctx.target_topics )
    _apply_time_scope(source_name, params, ctx)
    spec = SourceQuerySpec(
        source_name=source_name, query_text=query_text,
            target_topics=target_topics,
        max_results=ctx.max_results_per_call, strategy_name="agent",
            params=params,
    )
    key = source_query_key(spec)
    if key in ctx.executed_keys:
        return _text(f"Skipped: this exact {source_name} query already
            ran. Vary the operators or query text.")
    if ctx.adaptive and len(ctx.executed_calls) >=
        ctx.max_calls_this_round:
        return _text("Skipped: per-round source-call budget exhausted.
            Call submit_round_summary.")
    if not ctx.budget_remaining():
        return _text("Skipped: API-call budget exhausted for this run.
            Call submit_round_summary.")
    items, stat = search_source(
        source_name, query_text or " ".join(ctx.target_topics[:3]),
        target_topics=target_topics or ctx.target_topics,
        max_results=ctx.max_results_per_call, **params, )
    ctx.executed_keys.add(key)
    ctx.api_calls_used += 1
    ctx.stats.append(stat)
    new_ids: list[str] = []
    kept = 0
    duplicates = 0
    for item in items:
        if not ctx.in_time_scope(item):
            continue
        kept += 1
        if item.item_id in ctx.existing_item_ids or item.item_id in
            ctx.collected_items:
            duplicates += 1
            continue
        item.metadata["collection_strategy"] = "agent"
        item.metadata["source_query"] = query_text
        item.metadata["source_query_params"] = params
        ctx.collected_items[item.item_id] = item
        new_ids.append(item.item_id)
    has_more = source_name in {"nvd_cve_api", "arxiv_api"} and len(items)
        >= ctx.max_results_per_call
    call_id = f"{ctx.run_id}:call:{ctx.api_calls_used}"
    ctx.executed_calls.append( { "call_id": call_id,
            "candidate_id": candidate_id, "source_name": source_name,
            "query_text": query_text, "params": params,
```

```
            "query_intent": query_intent,
            "evidence_channel": evidence_channel,
            "target_topics": target_topics,
            "target_gap_ids": target_gap_ids, "new_item_ids": new_ids,
            "signature": operator_signature(source_name, params),
            "raw_count": len(items), "in_scope_count": kept,
            "duplicate_count": duplicates, "has_more": has_more,
            "success": stat.success, "error_type": stat.error_type,
            "latency_ms": stat.latency_ms, } )
    if candidate_id:
        ctx.approved_candidates.pop(candidate_id, None)
    if not stat.success:
        return _text(f"{source_name} error: {stat.error_type}. Try a
            different source or operators.")
    titles = "; ".join(item.title[:70] for item in
        list(ctx.collected_items.values())[-3:])
    return _text(
        f"{source_name}: fetched {len(items)} raw / {kept} in-scope, "
        f"{duplicates} duplicate, {len(new_ids)} new, has_more={has_more}.
            "
        f"operators={_fmt(params)}. Newest: {titles or '(none)'}" )
def build_source_tools(ctx: ToolContext) -> list[Any]:
    from claude_agent_sdk import tool
    @tool("search_nvd", "Search the NVD CVE API 2.0 for LLM/AI-related
        CVEs. "
          "Combine keyword_text with nvd_cwe_id / nvd_cvss_v3_severity /
              date window / nvd_has_kev "
          "to cut noise (e.g. CWE-502 unsafe deserialization, CWE-94 code
              injection).", _NVD_SCHEMA)
    async def search_nvd(args: dict[str, Any]) -> dict[str, Any]:
        return _run_call(ctx, "nvd_cve_api", args)
    @tool("search_arxiv", "Search arXiv for LLM-security papers. Prefer
        arxiv_search_query with the "
          "arXiv query language (field prefixes, AND/OR, quoted phrases,
              cat:cs.CR) for precision.",
         _ARXIV_SCHEMA)
    async def search_arxiv(args: dict[str, Any]) -> dict[str, Any]:
        return _run_call(ctx, "arxiv_api", args)
    @tool("search_cisa_kev", "Search the CISA Known Exploited
        Vulnerabilities catalog by keyword or "
          "specific CVE IDs. Best for confirming real-world exploited
              AI-stack CVEs.", _CISA_SCHEMA)
    async def search_cisa_kev(args: dict[str, Any]) -> dict[str, Any]:
        return _run_call(ctx, "cisa_kev_json", args)
    @tool("search_osv", "Search OSV.dev for open-source ecosystem
        advisories. Use osv_ecosystem + "
          "osv_package_name (e.g. PyPI/mlflow) or osv_vuln_id for direct
              lookup.", _OSV_SCHEMA)
    async def search_osv(args: dict[str, Any]) -> dict[str, Any]:
        return _run_call(ctx, "osv_dev_api", args)
```

```
    return [search_nvd, search_arxiv, search_cisa_kev, search_osv]
def _fmt(params: dict[str, Any]) -> str:
    return json.dumps(params, ensure_ascii=False) if params else "base"
def _text(message: str) -> dict[str, Any]:
    return {"content": [{"type": "text", "text": message}]}
# 文件 backend/intel_agent/sources.py
"""Registered intelligence source clients: NVD, arXiv, CISA KEV, OSV.dev.
Self-contained port of the legacy ``registered_source_tools`` HTTP/parse
    layer
with the CrewAI ``BaseTool`` wrapper removed (the agentic loop calls these
    plain
functions through ``@tool`` wrappers instead) and a small shared retry
    added to
the previously bare urllib calls.
"""
from __future__ import annotations
from intel_agent.runtime.client import _setting
import json
import re
import time
import xml.etree.ElementTree as ET
from datetime import datetime, timezone
from http.client import RemoteDisconnected
from typing import Any
from urllib.error import HTTPError, URLError
from urllib.parse import urlencode
from urllib.request import Request, urlopen
from .schemas import ApprovedSource, RawIntelItem, SourceExecutionStat
from .topics import detect_topics, tokenize
NVD_CVE_API_URL = "https://services.nvd.nist.gov/rest/json/cves/2.0"
ARXIV_API_URL = "https://export.arxiv.org/api/query"
CISA_KEV_JSON_URL = (
"https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabil
ities.json"
)
OSV_QUERY_URL = "https://api.osv.dev/v1/query"
OSV_VULN_URL = "https://api.osv.dev/v1/vulns"
REGISTERED_API_SOURCE_NAMES = {"nvd_cve_api", "arxiv_api",
    "cisa_kev_json", "osv_dev_api"}
USER_AGENT = "intel-agent/0.1"
ATOM_NAMESPACE = {"atom": "http://www.w3.org/2005/Atom"}
CVE_PATTERN = re.compile(r"CVE-\d{4}-\d{4,}", re.IGNORECASE)
ADVISORY_ID_PATTERN = re.compile(
r"(GHSA-[a-z0-9]{4}-[a-z0-9]{4}-[a-z0-9]{4}|PYSEC-\d{4}-\d+|RUSTSEC-\d{4}-
\d+)",
    re.IGNORECASE, )
_RETRYABLE = (URLError, TimeoutError, RemoteDisconnected, ConnectionError)
DEFAULT_OSV_PACKAGE_TARGETS = [
    {"ecosystem": "PyPI", "name": "langchain"},
    {"ecosystem": "PyPI", "name": "llama-index"},
```

```
    {"ecosystem": "PyPI", "name": "transformers"},
    {"ecosystem": "PyPI", "name": "gradio"},
    {"ecosystem": "PyPI", "name": "mlflow"},
    {"ecosystem": "PyPI", "name": "vllm"},
    {"ecosystem": "PyPI", "name": "open-webui"},
    {"ecosystem": "PyPI", "name": "langflow"},
    {"ecosystem": "PyPI", "name": "chromadb"},
    {"ecosystem": "PyPI", "name": "qdrant-client"},
    {"ecosystem": "PyPI", "name": "weaviate-client"},
    {"ecosystem": "PyPI", "name": "jupyter-server"},
    {"ecosystem": "PyPI", "name": "ray"},
    {"ecosystem": "npm", "name": "flowise"},
    {"ecosystem": "npm", "name": "langchain"}, ]
NVD_AI_ATTACK_KEYWORDS = [
    "prompt injection", "indirect prompt injection", "jailbreak",
    "large language model", "LLM", "chatbot", "model extraction",
    "model inversion", "data poisoning", "adversarial example",
    "training data", "RAG", "embedding", "vector database", ]
NVD_AI_PRODUCT_KEYWORDS = [
    "LangChain", "LlamaIndex", "Ollama", "vLLM", "llama.cpp", "Hugging
       Face",
    "Transformers", "MLflow", "Gradio", "Jupyter", "Ray", "Kubeflow",
    "TensorFlow", "PyTorch", "NVIDIA Triton", "ONNX", "Dify", "Langflow",
    "Flowise", "Milvus", "Qdrant", "Weaviate", "Chroma", ]
NVD_AI_RELEVANT_CWE_IDS = [
    "CWE-20", "CWE-22", "CWE-78", "CWE-79", "CWE-89", "CWE-94",
    "CWE-200", "CWE-287", "CWE-434", "CWE-502", "CWE-918", ]
def default_registered_api_sources() -> list[ApprovedSource]:
    return [ ApprovedSource( source_name="nvd_cve_api",
           base_uri=NVD_CVE_API_URL, source_type="security_db",
           trust_level=0.9,
           notes="NVD CVE API 2.0 (keywordSearch,
              pubStartDate/pubEndDate, hasKev, cveId).",
           metadata={"supports": ["keywordSearch", "pubStartDate",
              "pubEndDate", "hasKev"]},
       ), ApprovedSource( source_name="arxiv_api",
           base_uri=ARXIV_API_URL, source_type="paper", trust_level=0.72,
           notes="arXiv Atom API for LLM security papers and preprints.",
           metadata={"best_for": ["prompt injection", "jailbreak", "rag
              poisoning"]},
       ), ApprovedSource( source_name="cisa_kev_json",
           base_uri=CISA_KEV_JSON_URL, source_type="advisory",
           trust_level=0.92,
           notes="CISA Known Exploited Vulnerabilities JSON catalog.",
           metadata={"best_for": ["known exploited vulnerabilities",
              "kev"]},
       ), ApprovedSource( source_name="osv_dev_api",
           base_uri=OSV_QUERY_URL, source_type="security_db",
           trust_level=0.84,
           notes="OSV.dev API for open source ecosystem vulnerability
```

```
                intelligence.",
            metadata={"best_for": ["GitHub Advisory", "PyPI", "RustSec",
                "npm", "Go"]},
        ), ]
# ---------------------------------------------------------------------
    dispatch
def search_source( source_name: str, query_text: str,
    target_topics: list[str] | None = None, max_results: int = 10,
    timeout_seconds: int = 20, **params: Any,
) -> tuple[list[RawIntelItem], SourceExecutionStat]:
    """Run one source search; never raises -- failures return an error
        stat."""
    start = time.perf_counter()
    topics = target_topics or []
    try:
        if source_name == "nvd_cve_api":
            items = search_nvd_cve_api(
                query_text, max_results, timeout_seconds,
                keyword_search=params.get("nvd_keyword_search"),
                pub_start_date=params.get("nvd_pub_start_date"),
                pub_end_date=params.get("nvd_pub_end_date"),
                has_kev=bool(params.get("nvd_has_kev")),
                cve_id=params.get("nvd_cve_id"),
keyword_exact_match=bool(params.get("nvd_keyword_exact_match")),
                cwe_id=params.get("nvd_cwe_id"),
                cvss_v3_severity=params.get("nvd_cvss_v3_severity"),
                no_rejected=bool(params.get("nvd_no_rejected", True)),
                start_index=int(params.get("nvd_start_index") or 0), )
        elif source_name == "arxiv_api":
            items = search_arxiv_api(
                query_text, topics, max_results, timeout_seconds,
                arxiv_search_query=params.get("arxiv_search_query"),
                arxiv_start=int(params.get("arxiv_start") or 0), )
        elif source_name == "cisa_kev_json":
            items = search_cisa_kev_json(
                query_text, max_results, timeout_seconds,
                cve_ids=params.get("cisa_cve_ids") or [],
                keyword=params.get("cisa_keyword"), )
        elif source_name == "osv_dev_api":
            items = search_osv_dev_api(
                query_text, max_results, timeout_seconds,
                ecosystem=params.get("osv_ecosystem"),
                package_name=params.get("osv_package_name"),
                version=params.get("osv_version"),
                purl=params.get("osv_purl"),
                vuln_id=params.get("osv_vuln_id"), )
        else:
            items = []
        stat = SourceExecutionStat(
            source_name=source_name, query_count=1,
```

```
            result_count=len(items),
            success=True, latency_ms=_elapsed_ms(start), notes="ok", )
        return items, stat
    except (HTTPError, URLError, TimeoutError, RemoteDisconnected,
        ConnectionError,
            ValueError, ET.ParseError) as exc:
        stat = SourceExecutionStat(
            source_name=source_name, query_count=1, result_count=0,
                success=False,
            latency_ms=_elapsed_ms(start),
                error_type=exc.__class__.__name__, notes=str(exc),
        )
        return [], stat
# ---------------------------------------------------------------
    per-source
def search_nvd_cve_api(
    query_text: str, max_results: int, timeout_seconds: int,
    keyword_search: str | None = None, pub_start_date: str | None = None,
    pub_end_date: str | None = None, has_kev: bool = False, cve_id: str |
        None = None,
    keyword_exact_match: bool = False, cwe_id: str | None = None,
    cvss_v3_severity: str | None = None, no_rejected: bool = True,
    start_index: int = 0, ) -> list[RawIntelItem]:
    params = _build_nvd_query_params(
        query_text, max_results, keyword_search, pub_start_date,
            pub_end_date,
        has_kev, cve_id, keyword_exact_match, cwe_id, cvss_v3_severity,
            no_rejected,
        start_index, )
    data = _fetch_json(_url_with_params(NVD_CVE_API_URL, params),
        timeout_seconds)
    return parse_nvd_items(data)
def search_arxiv_api(
    query_text: str, target_topics: list[str], max_results: int,
    timeout_seconds: int, arxiv_search_query: str | None = None,
    arxiv_start: int = 0, ) -> list[RawIntelItem]:
    search_query = arxiv_search_query or
        build_arxiv_search_query(query_text, target_topics)
    params = {
        "search_query": search_query, "start": str(max(0, arxiv_start)),
            "max_results": str(max_results),
        "sortBy": "submittedDate", "sortOrder": "descending", }
    raw_xml = _fetch_text(_url_with_params(ARXIV_API_URL, params),
        timeout_seconds)
    return parse_arxiv_items(raw_xml)
def search_cisa_kev_json(
    query_text: str, max_results: int, timeout_seconds: int,
    cve_ids: list[str] | None = None, keyword: str | None = None,
) -> list[RawIntelItem]:
    data = _fetch_json(CISA_KEV_JSON_URL, timeout_seconds)
```

```
    return parse_cisa_kev_items(data, query_text, cve_ids or [], keyword,
        max_results)
def search_osv_dev_api(
    query_text: str, max_results: int, timeout_seconds: int,
    ecosystem: str | None = None, package_name: str | None = None,
    version: str | None = None, purl: str | None = None, vuln_id: str |
        None = None,
) -> list[RawIntelItem]:
    direct_id = vuln_id or _extract_first_vulnerability_id(query_text)
    if direct_id:
        data = _fetch_json(f"{OSV_VULN_URL}/{direct_id}", timeout_seconds)
        return parse_osv_vulnerability(data)[:max_results]
    payload = build_osv_query_payload(ecosystem, package_name, version,
        purl)
    payloads = [payload] if payload else [{"package": pkg} for pkg in
        DEFAULT_OSV_PACKAGE_TARGETS]
    items: list[RawIntelItem] = []
    seen: set[str] = set()
    for next_payload in payloads:
        data = _fetch_json(OSV_QUERY_URL, timeout_seconds, method="POST",
            payload=next_payload)
        for item in parse_osv_query_response(data, query_text):
            if item.item_id in seen:
                continue
            seen.add(item.item_id)
            items.append(item)
            if len(items) >= max_results:
                return items
    return items
# ------------------------------------------------------------ query
    builders
def _build_nvd_query_params(
    query_text: str, max_results: int, keyword_search: str | None,
    pub_start_date: str | None, pub_end_date: str | None, has_kev: bool,
    cve_id: str | None, keyword_exact_match: bool, cwe_id: str | None,
    cvss_v3_severity: str | None, no_rejected: bool, start_index: int = 0,
) -> dict[str, str | None]:
    cve = cve_id or _extract_first_cve_id(query_text)
    params: dict[str, str | None] = {
        "resultsPerPage": str(max_results), "startIndex": str(max(0,
            start_index))
    }
    if cve:
        params["cveId"] = cve
    else:
        params["keywordSearch"] = keyword_search or
            _compact_keyword_query(query_text)
        if keyword_exact_match and " " in params["keywordSearch"]:
            params["keywordExactMatch"] = None
    if cwe_id:
```

```
        params["cweId"] = cwe_id
    if cvss_v3_severity:
        params["cvssV3Severity"] = cvss_v3_severity.upper()
    if pub_start_date and pub_end_date:
        params["pubStartDate"] = pub_start_date
        params["pubEndDate"] = pub_end_date
    if has_kev:
        params["hasKev"] = None
    if no_rejected:
        params["noRejected"] = None
    return params
def build_arxiv_search_query(query_text: str, target_topics: list[str]) ->
    str:
    phrases = target_topics or detect_topics(query_text) or [
        "prompt injection", "jailbreak", "RAG poisoning", "large language
            model security",
    ]
    phrase_query = " OR ".join(f'all:"{phrase}"' for phrase in
        phrases[:6])
    return f"({phrase_query}) AND (cat:cs.CR OR cat:cs.AI OR cat:cs.CL)"
def build_osv_query_payload(
    ecosystem: str | None = None, package_name: str | None = None,
    version: str | None = None, purl: str | None = None,
) -> dict[str, Any]:
    if purl:
        payload: dict[str, Any] = {"package": {"purl": purl}}
    elif ecosystem and package_name:
        payload = {"package": {"ecosystem": ecosystem, "name":
            package_name}}
    else:
        return {}
    if version and "@" not in payload["package"].get("purl", ""):
        payload["version"] = version
    return payload
# ---------------------------------------------------------------------
    parsers
def parse_nvd_items(data: dict[str, Any]) -> list[RawIntelItem]:
    items: list[RawIntelItem] = []
    for entry in data.get("vulnerabilities", []):
        cve = entry.get("cve", {})
        cve_id = cve.get("id", "")
        if not cve_id:
            continue
        description = _first_english_description(cve.get("descriptions",
            []))
        references = _extract_nvd_references(cve)
        primary_ref = references[0].get("url") if references else
            f"{NVD_CVE_API_URL}?cveId={cve_id}"
        cwe_ids = _extract_nvd_cwe_ids(cve)
        cpe_products = _extract_nvd_cpe_products(cve)
```

```
        nvd_text = " ".join([description, " ".join(cpe_products)])
        topics = detect_topics(nvd_text)
        ai_category, matched_terms = classify_ai_vulnerability(nvd_text,
            cwe_ids)
        relevance = estimate_relevance(nvd_text, topics)
        if ai_category != "uncategorized":
            relevance = max(relevance, 0.68)
        items.append( RawIntelItem( item_id=f"nvd:{cve_id.lower()}",
                source_name="nvd_cve_api", source_uri=primary_ref,
                title=f"{cve_id}: {_shorten(description, 120)}",
                summary=description,
                published_at=_parse_datetime(cve.get("published")),
                raw_text=description, relevance_score=relevance,
                extraction_notes="Parsed from NVD CVE API 2.0 response.",
                metadata={
                    "cve_id": cve_id, "topics": topics,
                "ai_intel_category": ai_category,
                    "matched_ai_terms": matched_terms, "cwe_ids": cwe_ids,
                    "cvss_v3_severity":
                _extract_nvd_cvss_v3_severity(cve),
                    "cpe_products": cpe_products, "source_type":
                "security_db",
                }, ) )
    return items
def parse_arxiv_items(raw_xml: str) -> list[RawIntelItem]:
    root = ET.fromstring(raw_xml)
    items: list[RawIntelItem] = []
    for entry in root.findall("atom:entry", ATOM_NAMESPACE):
        item_id = _node_text(entry, "atom:id")
        title = _clean_text(_node_text(entry, "atom:title"))
        summary = _clean_text(_node_text(entry, "atom:summary"))
        if title.lower() == "error":
            continue
        topics = detect_topics(f"{title} {summary}")
        items.append( RawIntelItem(
                item_id=f"arxiv:{item_id.rsplit('/', 1)[-1].lower()}",
                source_name="arxiv_api", source_uri=item_id, title=title,
                summary=summary,
                published_at=_parse_datetime(_node_text(entry,
                "atom:published")),
                raw_text=summary,
                relevance_score=estimate_relevance(f"{title} {summary}",
                topics),
                extraction_notes="Parsed from arXiv Atom API response.",
                metadata={ "topics": topics, "source_type": "paper",
                    "authors": [ _node_text(author, "atom:name")
                        for author in entry.findall("atom:author",
                ATOM_NAMESPACE)
                    ], }, ) )
    return items
```

```
def parse_cisa_kev_items(
    data: dict[str, Any], query_text: str, cve_ids: list[str],
    keyword: str | None, max_results: int, ) -> list[RawIntelItem]:
    wanted = {cve_id.upper() for cve_id in cve_ids}
    wanted.update(match.upper() for match in
       CVE_PATTERN.findall(query_text))
    query_tokens = tokenize(keyword or query_text)
    items: list[RawIntelItem] = []
    for entry in data.get("vulnerabilities", []):
       cve_id = str(entry.get("cveID", "")).upper()
       text = " ".join( str(entry.get(key, ""))
          for key in ("vendorProject", "product", "vulnerabilityName",
                    "shortDescription", "requiredAction",
             "knownRansomwareCampaignUse")
       )
       if wanted and cve_id not in wanted:
          continue
       if not wanted and query_tokens and not (query_tokens &
          tokenize(text)):
          continue
       topics = detect_topics(text)
       items.append( RawIntelItem( item_id=f"cisa-kev:{cve_id.lower()}",
             source_name="cisa_kev_json", source_uri=(
"https://www.cisa.gov/known-exploited-vulnerabilities-catalog"
                f"?search_api_fulltext={cve_id}" ),
             title=f"{cve_id}: {entry.get('vulnerabilityName', '')}",
             summary=str(entry.get("shortDescription", "")),
             published_at=_parse_date(entry.get("dateAdded")),
             raw_text=text,
             relevance_score=max(0.55, estimate_relevance(text,
             topics)),
             extraction_notes="Parsed from CISA KEV JSON catalog.",
             metadata={ "cve_id": cve_id, "topics": topics,
                "vendor_project": entry.get("vendorProject"),
                "product": entry.get("product"), "due_date":
             entry.get("dueDate"),
                "date_added": entry.get("dateAdded"), "source_type":
             "advisory",
             }, ) )
       if len(items) >= max_results:
          break
    return items
def parse_osv_query_response(data: dict[str, Any], query_text: str) ->
    list[RawIntelItem]:
    items: list[RawIntelItem] = []
    for vuln in data.get("vulns", []):
       items.extend(parse_osv_vulnerability(vuln, query_text))
    return items
def parse_osv_vulnerability(data: dict[str, Any], query_text: str = "") ->
    list[RawIntelItem]:
```

```
    vuln_id = str(data.get("id", ""))
    if not vuln_id:
        return []
    summary = str(data.get("summary") or data.get("details") or "")
    details = str(data.get("details") or summary)
    affected = data.get("affected", [])
    references = data.get("references", [])
    source_uri = references[0].get("url") if references else
        f"{OSV_VULN_URL}/{vuln_id}"
    topics = detect_topics(f"{summary} {details} {query_text}")
    ecosystems = sorted( { pkg.get("ecosystem", "") for item in affected
           for pkg in [item.get("package", {})] if pkg.get("ecosystem") }
    )
    return [ RawIntelItem( item_id=f"osv:{vuln_id.lower()}",
           source_name="osv_dev_api", source_uri=source_uri,
           title=f"{vuln_id}: {_shorten(summary, 160)}",
           summary=summary or _shorten(details, 300),
           published_at=_parse_datetime(data.get("published") or
               data.get("modified")),
           raw_text=details,
           relevance_score=estimate_relevance(f"{summary} {details}
               {query_text}", topics),
           extraction_notes="Parsed from OSV.dev vulnerability API
               response.",
           metadata={
               "vulnerability_id": vuln_id, "aliases":
               data.get("aliases", []),
               "topics": topics, "ecosystems": ecosystems,
               "modified": data.get("modified"), "source_type":
               "security_db",
           }, ) ]
# ---------------------------------------------------------------------
    http
def _fetch_json( url: str, timeout_seconds: int, method: str = "GET",
    payload: dict[str, Any] | None = None, retries: int = 2,
) -> dict[str, Any]:
    data = None
    headers = {"Accept": "application/json", "User-Agent": USER_AGENT}
    if payload is not None:
        data = json.dumps(payload).encode("utf-8")
        headers["Content-Type"] = "application/json"
    nvd_key = _setting("NVD_API_KEY")
    if nvd_key and url.startswith(NVD_CVE_API_URL):
        headers["apiKey"] = nvd_key
    raw = _request_with_retry(url, headers, data, method, timeout_seconds,
        retries)
    return json.loads(raw)
def _fetch_text(url: str, timeout_seconds: int, retries: int = 2) -> str:
    return _request_with_retry(
        url, {"User-Agent": USER_AGENT}, None, "GET", timeout_seconds,
```

```
            retries
    )
def _request_with_retry(
    url: str, headers: dict[str, str], data: bytes | None,
    method: str, timeout_seconds: int, retries: int, ) -> str:
    last_exc: Exception | None = None
    for attempt in range(retries + 1):
        try:
            request = Request(url, data=data, headers=headers,
                method=method)
            with urlopen(request, timeout=timeout_seconds) as response:
                return response.read().decode("utf-8")
        except HTTPError as exc:
            if exc.code in (429, 500, 502, 503, 504) and attempt <
                retries:
                last_exc = exc
                time.sleep(1.5 * (attempt + 1))
                continue
            raise
        except _RETRYABLE as exc:
            last_exc = exc
            if attempt < retries:
                time.sleep(1.5 * (attempt + 1))
                continue
            raise
    raise last_exc if last_exc else RuntimeError("request failed without
        exception")
def _url_with_params(base_url: str, params: dict[str, str | None]) -> str:
    parts = [key if value is None else urlencode({key: value}) for key,
        value in params.items()]
    return f"{base_url}?{'&'.join(parts)}"
# ---------------------------------------------------------------------
    helpers
def estimate_relevance(text: str, topics: list[str]) -> float:
    topic_bonus = min(0.3, 0.1 * len(topics))
    token_bonus = min(0.2, 0.02 * len(tokenize(text)))
    return min(1.0, 0.45 + topic_bonus + token_bonus)
def classify_ai_vulnerability(text: str, cwe_ids: list[str]) -> tuple[str,
    list[str]]:
    lower = text.lower()
    attack = [term for term in NVD_AI_ATTACK_KEYWORDS if term.lower() in
        lower]
    product = [term for term in NVD_AI_PRODUCT_KEYWORDS if term.lower() in
        lower]
    cwe = [cwe_id for cwe_id in cwe_ids if cwe_id in
        NVD_AI_RELEVANT_CWE_IDS]
    matched = sorted(set(attack + product + cwe))
    if attack:
        return "ai_native_attack", matched
    if product and cwe:
```

```
        return "ai_application_infrastructure_vulnerability", matched
    if product:
        return "ai_data_model_supply_chain", matched
    return "uncategorized", matched
def _compact_keyword_query(query_text: str) -> str:
    topics = detect_topics(query_text)
    if topics:
        return " ".join(topics[:3])
    tokens = sorted(tokenize(query_text))
    return " ".join(tokens[:4]) or "large language model"
def _extract_first_vulnerability_id(query_text: str) -> str | None:
    cve_id = _extract_first_cve_id(query_text)
    if cve_id:
        return cve_id
    match = ADVISORY_ID_PATTERN.search(query_text)
    return match.group(0).upper() if match else None
def _extract_first_cve_id(query_text: str) -> str | None:
    match = CVE_PATTERN.search(query_text)
    return match.group(0).upper() if match else None
def _extract_nvd_cwe_ids(cve: dict[str, Any]) -> list[str]:
    cwe_ids: set[str] = set()
    for weakness in cve.get("weaknesses", []):
        for description in weakness.get("description", []):
            value = str(description.get("value", ""))
            if value.startswith("CWE-"):
                cwe_ids.add(value)
    return sorted(cwe_ids)
def _extract_nvd_references(cve: dict[str, Any]) -> list[dict[str, Any]]:
    raw = cve.get("references", [])
    if isinstance(raw, dict):
        data = raw.get("referenceData", [])
        return data if isinstance(data, list) else []
    if isinstance(raw, list):
        return [item for item in raw if isinstance(item, dict)]
    return []
def _extract_nvd_cvss_v3_severity(cve: dict[str, Any]) -> str | None:
    metrics = cve.get("metrics", {})
    for key in ("cvssMetricV31", "cvssMetricV30"):
        values = metrics.get(key) or []
        if values:
            severity = values[0].get("cvssData", {}).get("baseSeverity")
            if severity:
                return str(severity)
    return None
def _extract_nvd_cpe_products(cve: dict[str, Any]) -> list[str]:
    products: set[str] = set()
    for configuration in cve.get("configurations", []):
        for node in configuration.get("nodes", []):
            for match in node.get("cpeMatch", []):
                parts = str(match.get("criteria", "")).split(":")
```

```
            if len(parts) >= 6:
                vendor = parts[3].replace("_", " ")
                product = parts[4].replace("_", " ")
                products.add(f"{vendor} {product}".strip())
    return sorted(products)
def _first_english_description(descriptions: list[dict[str, Any]]) -> str:
    for description in descriptions:
        if description.get("lang") == "en":
            return str(description.get("value", ""))
    return str(descriptions[0].get("value", "")) if descriptions else ""
def _parse_datetime(value: Any) -> datetime | None:
    if not value:
        return None
    raw = str(value).replace("Z", "+00:00")
    try:
        parsed = datetime.fromisoformat(raw)
    except ValueError:
        return _parse_date(raw)
    return parsed.replace(tzinfo=timezone.utc) if parsed.tzinfo is None
        else parsed.astimezone(timezone.utc)
def _parse_date(value: Any) -> datetime | None:
    if not value:
        return None
    try:
        return datetime.strptime(str(value),
            "%Y-%m-%d").replace(tzinfo=timezone.utc)
    except ValueError:
        return None
def _node_text(node: ET.Element, path: str) -> str:
    found = node.find(path, ATOM_NAMESPACE)
    return found.text if found is not None and found.text is not None else
        ""
def _clean_text(text: str) -> str:
    return re.sub(r"\s+", " ", text).strip()
def _shorten(text: str, max_length: int) -> str:
    cleaned = _clean_text(text)
    return cleaned if len(cleaned) <= max_length else f"{cleaned[:
        max_length - 3].rstrip()}..."
def _elapsed_ms(start: float) -> float:
    return round((time.perf_counter() - start) * 1000, 2)
# 文件 backend/intel_agent/persistence.py
"""File-backed store for intel_agent runs.
Writes ``data/intel_runs/<run_id>.json`` + ``latest.json`` in a layout
    that the
legacy Gradio console can still read (extra keys tolerated), while
    carrying no
import dependency on the legacy package. KG bookkeeping is intentionally
dropped: the intel_agent engine does not generate knowledge graphs.
"""
from __future__ import annotations
```

```
import json
import re
from datetime import datetime, timezone
from pathlib import Path
from typing import Any
from .schemas import IntelRunBlackboard, RawIntelItem, RawIntelItemBatch,
   SourceCheckpoint
from .topics import EXTENDED_SECURITY_TOPICS
DEFAULT_INTEL_RUN_DIR = Path("data") / "intel_runs"
LATEST_INDEX_NAME = "latest.json"
class JsonIntelRunStore:
   def __init__( self, root_dir: str | Path = DEFAULT_INTEL_RUN_DIR,
   ) -> None:
      self.root_dir = Path(root_dir)
   def run_path(self, run_id: str) -> Path:
      return self.root_dir / f"{_safe_run_id(run_id)}.json"
   def latest_index_path(self) -> Path:
      return self.root_dir / LATEST_INDEX_NAME
   def save_run( self, blackboard: IntelRunBlackboard,
      raw_item_batches: list[RawIntelItemBatch] | None = None,
      status: str = "succeeded", ) -> Path:
      self.root_dir.mkdir(parents=True, exist_ok=True)
      saved_at = _utcnow_iso()
      path = self.run_path(blackboard.run_id)
      payload = { "schema_version": 3, "run_id": blackboard.run_id,
         "status": status, "saved_at": saved_at,
         "engine": "intel_agent",
         "summary": self._summary(blackboard, raw_item_batches or []),
         "raw_item_batches": [b.model_dump(mode="json") for b in
            raw_item_batches or []],
         "blackboard": blackboard.model_dump(mode="json"), }
      tmp_path = path.with_suffix(".json.tmp")
      tmp_path.write_text(json.dumps(payload, ensure_ascii=False,
         indent=2), encoding="utf-8")
      tmp_path.replace(path)
      self.latest_index_path().write_text( json.dumps( {
               "run_id": blackboard.run_id, "status": status,
               "path": path.name, "saved_at": saved_at,
               "raw_items": len(blackboard.raw_items),
               "rounds": len(blackboard.query_history), },
            ensure_ascii=False, indent=2, ), encoding="utf-8", )
      return path
   def load_run_payload(self, run_id: str) -> dict[str, Any]:
      return
         json.loads(self.run_path(run_id).read_text(encoding="utf-8"))
   def load_latest_payload(self) -> dict[str, Any] | None:
      index_path = self.latest_index_path()
      if not index_path.exists():
         return None
      index = json.loads(index_path.read_text(encoding="utf-8"))
```

```
        name = index.get("path")
        if not name:
            return None
        path = self.root_dir / str(name)
        return json.loads(path.read_text(encoding="utf-8")) if
            path.exists() else None
    def list_run_paths(self, limit: int = 20) -> list[Path]:
        if not self.root_dir.exists():
            return []
        paths = [p for p in self.root_dir.glob("*.json") if p.name !=
            LATEST_INDEX_NAME]
        paths.sort(key=lambda p: p.stat().st_mtime, reverse=True)
        return paths[:limit]
    def load_all_blackboards(self, limit: int = 200) ->
        list[IntelRunBlackboard]:
        boards: list[IntelRunBlackboard] = []
        for path in self.list_run_paths(limit=limit):
            try:
                payload = json.loads(path.read_text(encoding="utf-8"))
boards.append(IntelRunBlackboard.model_validate(payload["blackboard"]))
            except (OSError, json.JSONDecodeError, KeyError, ValueError):
                continue
        return boards
    def load_corpus_items(self) -> list[RawIntelItem]:
        """Load the complete JSON corpus once, de-duplicated by natural
            item id."""
        by_id: dict[str, RawIntelItem] = {}
        for blackboard in
            reversed(self.load_all_blackboards(limit=100_000)):
            for item in blackboard.raw_items:
                by_id[item.item_id] = item
        return list(by_id.values())
    def load_source_checkpoints(self) -> list[SourceCheckpoint]:
        latest: dict[str, SourceCheckpoint] = {}
        for blackboard in self.load_all_blackboards(limit=100_000):
            for checkpoint in blackboard.source_checkpoints:
                latest.setdefault(checkpoint.source_name, checkpoint)
        return list(latest.values())
    def format_latest_intel(self, limit: int = 20) -> str:
        payload = self.load_latest_payload()
        if not payload:
            return f"No persisted intelligence runs found in
                {self.root_dir}."
        blackboard =
            IntelRunBlackboard.model_validate(payload["blackboard"])
        items = sorted( blackboard.raw_items,
            key=lambda item: item.published_at or item.fetched_at,
            reverse=True, )[:limit]
        lines = [ f"Latest run: {payload['run_id']}",
            f"Status: {payload.get('status', 'unknown')}",
```

```
            f"Saved at: {payload.get('saved_at', 'unknown')}", "",
            format_intel_items(items), ]
        return "\n".join(lines).rstrip()
    def store_backend(self) -> str:
        return f"json:{self.root_dir}"
    def run_count(self) -> int:
        return len(self.list_run_paths(limit=10_000))
    def distinct_item_count(self) -> int:
        return len(self.query_items(limit=100_000))
    def list_run_summaries(self, limit: int = 50) -> list[dict[str, Any]]:
        rows: list[dict[str, Any]] = []
        for path in self.list_run_paths(limit=limit):
            try:
                payload = json.loads(path.read_text(encoding="utf-8"))
            except (OSError, json.JSONDecodeError):
                continue
            summary = payload.get("summary") or {}
            bb = payload.get("blackboard") or {}
            rows.append( { "run_id": payload.get("run_id"),
                    "status": payload.get("status"),
                    "saved_at": payload.get("saved_at"),
                    "item_count": summary.get("raw_items",
                len(bb.get("raw_items") or [])),
                    "run_mode": summary.get("run_mode") or
                bb.get("run_mode"),
                    "rounds": summary.get("rounds",
                len(bb.get("query_history") or [])),
                    "run_goal": summary.get("run_goal") or
                bb.get("run_goal"),
                    "coverage_gaps": ",
                ".join(summary.get("coverage_gaps_remaining") or []),
                    "stop_reason": summary.get("stop_reason"),
                    "core_gap_count": summary.get("core_gap_count", 0),
                    "extended_item_count":
                summary.get("extended_item_count", 0),
                    "candidate_topic_count":
                summary.get("candidate_topic_count", 0),
                    "engine": payload.get("engine"), } )
        return rows
    def query_items( self, *, source_name: str | None = None,
        topic: str | None = None, text: str | None = None,
        run_id: str | None = None, limit: int = 200,
    ) -> list[dict[str, Any]]:
        """Flatten items from JSON run files (best-effort dedup by
            item_id)."""
        source = (source_name or "").strip().lower()
        topic_f = (topic or "").strip().lower()
        needle = (text or "").strip().lower()
        run_f = (run_id or "").strip()
        by_id: dict[str, dict[str, Any]] = {}
```

```
        paths = self.list_run_paths(limit=500)
        for path in paths:
            try:
                payload = json.loads(path.read_text(encoding="utf-8"))
            except (OSError, json.JSONDecodeError):
                continue
            rid = str(payload.get("run_id") or "")
            if run_f and rid != run_f:
                continue
            for raw in (payload.get("blackboard") or {}).get("raw_items")
                or []:
                item_id = str(raw.get("item_id") or "")
                if not item_id:
                    continue
                topics = list((raw.get("metadata") or {}).get("topics") or
                []))
                doc = { **raw, "item_id": item_id, "topics": topics,
                    "run_ids": [rid] if rid else [],
                    "last_run_id": rid or None, }
                if item_id in by_id:
                    existing = by_id[item_id]
                    run_ids = list(existing.get("run_ids") or [])
                    if rid and rid not in run_ids:
                        run_ids.append(rid)
                    existing["run_ids"] = run_ids
                    existing["last_run_id"] = rid or
                existing.get("last_run_id")
                else:
                    by_id[item_id] = doc
        rows: list[dict[str, Any]] = []
        for doc in by_id.values():
            if source and str(doc.get("source_name") or "").lower() !=
                source:
                continue
            topics = [str(t).lower() for t in (doc.get("topics") or [])]
            if topic_f and topic_f not in topics:
                continue
            if needle:
                blob = " ".join( [ str(doc.get("item_id") or ""),
                       str(doc.get("title") or ""),
                       str(doc.get("summary") or ""), ] ).lower()
                if needle not in blob:
                    continue
            rows.append(doc)
            if len(rows) >= max(1, int(limit)):
                break
        return rows
    def distinct_sources(self) -> list[str]:
        sources: set[str] = set()
        for doc in self.query_items(limit=10_000):
```

```
            name = doc.get("source_name")
            if name:
                sources.add(str(name))
        return sorted(sources)
    def get_item(self, item_id: str) -> dict[str, Any] | None:
        for doc in self.query_items(limit=10_000):
            if str(doc.get("item_id") or doc.get("_id")) == item_id:
                return doc
        return None
    def _summary(
        self, blackboard: IntelRunBlackboard, batches:
            list[RawIntelItemBatch]
    ) -> dict[str, Any]:
        return { "run_goal": blackboard.run_goal,
            "run_mode": blackboard.run_mode,
            "rounds": len(blackboard.query_history),
            "raw_items": len(blackboard.raw_items),
            "raw_item_batches": len(batches), "coverage_gaps_remaining": [
                gap.taxonomy_or_component for gap in
                blackboard.coverage_gaps
            ], "stop_reason": blackboard.stop_reason,
            "core_gap_count": sum(1 for gap in blackboard.corpus_gaps if
                gap.status == "open"),
            "extended_item_count": sum( 1 for item in blackboard.raw_items
                if set(item.metadata.get("topics") or []) &
                set(EXTENDED_SECURITY_TOPICS)
            ), "candidate_topic_count": len(blackboard.candidate_topics),
            "last_action": ( blackboard.action_history[-1].action_type
                if blackboard.action_history else None ), }
def format_intel_items(items: list[RawIntelItem]) -> str:
    if not items:
        return "No raw intelligence items found."
    lines: list[str] = []
    for index, item in enumerate(items, start=1):
        topics = ", ".join(item.metadata.get("topics", [])) or
            "unclassified"
        published = item.published_at.isoformat() if item.published_at
            else "unknown date"
        lines.append( "\n".join([
                f"{index}. {item.title or '(untitled)'}",
                f"   source: {item.source_name}",
                f"   published: {published}",
                f"   relevance: {item.relevance_score:.2f}",
                f"   topics: {topics}", f"   uri: {item.source_uri}", ]) )
    return "\n\n".join(lines)
def _safe_run_id(run_id: str) -> str:
    return re.sub(r"[^a-zA-Z0-9_.-]+", "_", run_id).strip("._") or
        "intel_run"
def _utcnow_iso() -> str:
    return datetime.now(timezone.utc).isoformat()
```